

Concurrency control is another issue in parallel and distributed databases. Since a transaction may access data items at several nodes, transaction managers at several nodes may need to coordinate to implement concurrency control. If locking is used, locking can be performed locally at the nodes containing accessed data items, but there is also a possibility of deadlock involving transactions originating at multiple nodes. Therefore deadlock detection needs to be carried out across multiple nodes. Failures are more common in distributed systems since not only may computers fail, but communication links may also fail. Replication of data items, which is the key to the continued functioning of distributed databases when failures occur, further complicates concurrency control. We describe concurrency-control techniques for distributed databases in Section 23.3 (which describes techniques based on locking) and Section 23.3.4 (which describes techniques based on timestamps).

The standard transaction models, based on multiple actions carried out by a single program unit, are often inappropriate for carrying out tasks that cross the boundaries of databases that cannot or will not cooperate to implement protocols such as 2PC. Alternative approaches, based on *persistent messaging* for communication, are generally used for such tasks; persistent messaging is discussed in Section 23.2.3.

When the tasks to be carried out are complex, involving multiple databases and/or multiple interactions with humans, coordination of the tasks and ensuring transaction properties for the tasks become more complicated. *Workflow management systems* are systems designed to help with carrying out such tasks.

20.7 Cloud-Based Services

Traditionally, enterprises purchased and ran servers that execute the database as well as the applications. There is a high cost to maintaining servers, including setting up server room infrastructure dealing with all kinds of failures such as air conditioning and power failures, not to mention failures of CPUs, disks, and other components of the servers. Further, if there is a sudden increase in demand, it is very difficult to add infrastructure to service the demand, and if demand falls, the infrastructure may lie idle.

In contrast, in the **cloud computing** model, applications of an enterprise are executed on an infrastructure that is managed by another company, typically at a data center that hosts a large number of machines used by many different enterprises/users. The service provider may provide not just hardware, but also support platforms such as databases, and application software.

A variety of vendors offer cloud services; these include major vendors such as Amazon, Microsoft, IBM, and Google, and a number of smaller vendors. One of the pioneers of cloud services, Amazon, originally built a large computing infrastructure purely for its internal use; then, seeing a business opportunity, it offered computing infrastructure as a service to other users. Cloud services became very popular within just a few years.

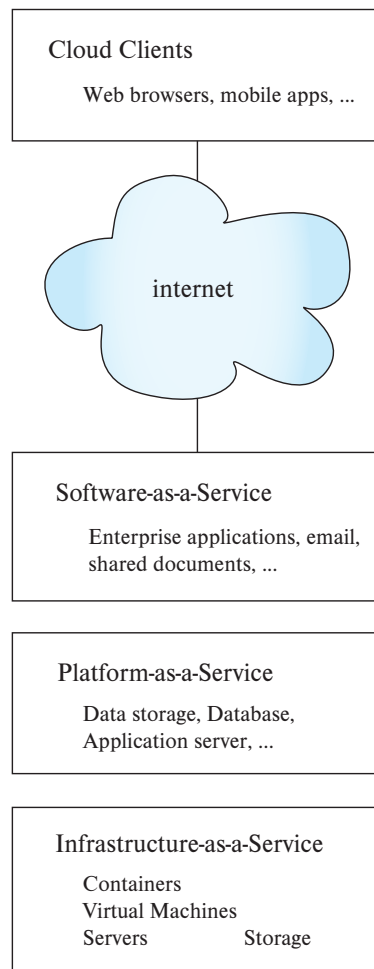


Figure 20.10 Cloud service models.

20.7.1 Cloud Service Models

There are several ways in which cloud computing can be utilized, which are summarized in Figure 20.10. These include infrastructure-as-a-service, platform-as-a-service, and software-as-a-service models.

- In the **infrastructure-as-a-service** model, an enterprise rents computing facilities; for example, an enterprise may rent one or more physical machines, along with disk storage space.

More frequently, cloud computing providers provide an abstraction of a **virtual machine (VM)**, which appears to the user to be a real machine. These machines are not “real” machines, but rather are simulated by software that allows a single

real computer to simulate several independent computers. *Containers* are a lower cost alternative to VMs and are described later in this section. Multiple VMs can run on a single server machine, and multiple containers can run on a single VM or server.

By running a very large data center with many machines, cloud-service providers can exploit economies of scale and deliver computing power at much lower cost than an enterprise can do using its own infrastructure.

Another major advantage of cloud computing is that the cloud-service provider usually has a large number of machines, with spare capacity, and thus an enterprise can rent more (virtual) machines as needed to meet demand and release them at times of light load. The ability to expand or contract capacity at short notice is often referred to as **elasticity**.

The above benefits of on-demand elastic provisioning of server systems have led to the widespread adoption of infrastructure-as-service platforms, especially by companies that anticipate rapid growth in their computing usage. However, due to the potential security risks of storing data outside the enterprise, the use of cloud computing is still limited in high-security enterprise needs, such as banking.

In the infrastructure-as-service model, the client enterprise runs its own software, including database systems, on virtual machines provided by the cloud-service provider; the client has to install the database system and deal with maintenance issues such as backup and restore.

- In the **platform-as-a-service** model, the service provider not only provides computing infrastructure, but it also deploys and manages platforms, such as data storage, databases, and application servers, that are used by application software. The client has to install and maintain application software, such as *enterprise resource planning (ERP)* systems, which run on such platform-provided services as application servers, database services, or data storage services.
 - **Cloud-based data storage** platforms provide a service that applications can use to store and retrieve data. The service provider takes care of provisioning sufficient amount of storage and computing power to support the load on the data storage platform. Such storage systems could support files, which are typically large, ranging in size from a few megabytes to thousands of megabytes, supporting millions of such files. Or such storage systems could support data items, which are typically small, ranging from hundreds of bytes to a few megabytes, but supporting billions of such data items. Such distributed file systems and data storage systems are discussed in Section 21.6 and Section 21.7. Database applications using cloud-based storage may run on the same cloud (i.e., the same set of machines), or on another cloud.

One of the main attractions of cloud-based storage is that it can be used by paying a fee without worrying about purchasing, maintaining, and managing the computer systems on which such a service runs. Further, if there is an

increase in demand, the number of servers on which the service runs can be increased by paying a larger fee, without having to actually purchase and deploy more servers. The service provider would of course have to deploy extra servers, but they benefit from economies of scale; the cost of deployment, and especially the time to deployment, are greatly reduced compared to what they would be if the end-users did it on their own.

The fees for cloud-based data storage are typically based on the amount of data stored, and amount of data input to, and the amount of data output from, the data storage system.

- **Database-as-a-service** platforms provide a database that can be accessed and queried by clients. Unlike storage services, database-as-a-service platforms provide database functionality such as querying using SQL or other query languages, which data storage systems do not provide. Early offerings of database-as-a-service only supported databases that run on a single node, although the node itself can have a substantial number of processors, memory, and storage. More recently, parallel database systems are being offered as a service on the cloud.
- In the **software-as-a-service** model, the service provider provides the application software as a service. The client does not need to deal with issues such as software installation or upgrades; these tasks are left to the service provider. The client can directly use interfaces provided by the software-as-a-service provider, such as web interfaces, or mobile app interfaces that provide a front end, with the application software acting as the back end.

The concept of virtual machines was developed in the 1960s to allow an expensive mainframe computer to be shared concurrently by users running different operating systems. Although computers are now much cheaper, there is still a cost associated with supplying electrical power to the computers and maintaining them; virtual machines allow this cost to be shared by multiple concurrent users. Virtual machines also ease the task of moving services to new machines: a virtual machine can be shut down on one physical server and restarted on another physical server with very little delay or downtime. This feature is particularly important for quick recovery in the event of hardware failure or upgrade.

Although multiple virtual machines can run on a single real machine, each VM has a high overhead, since it runs an entire operating system internally. When a single organization wishes to run a number of services, if it creates a separate VM for each service, the overhead can be very high. If multiple applications are run on one machine (or VM), two problems often arise: (1) applications conflict on network ports by each trying to listen to the same network port, and (2) applications require different versions of shared libraries, causing conflicts.

Containers solve both these problems; applications run in a container, which has its own IP address and its own set of shared libraries. Each application can consist of

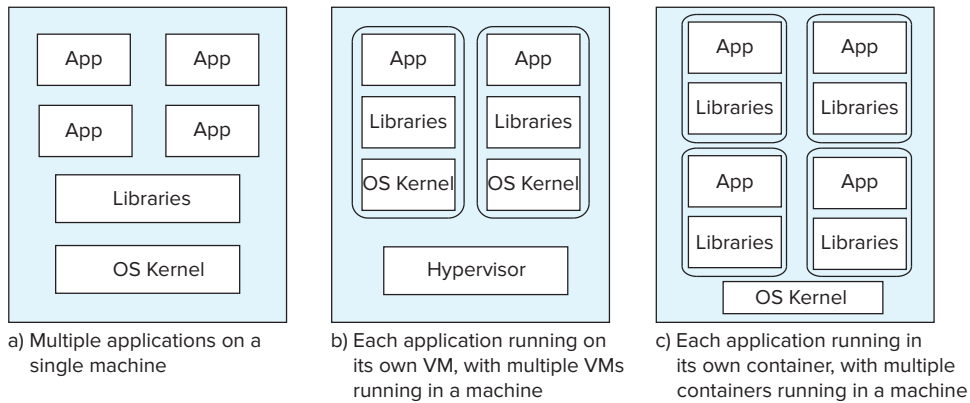


Figure 20.11 Application deployment alternatives.

multiple processes, all running within the same container. The cost of using containers to run applications is much less than the alternative of running each application in its own VM, since many containers can share the same operating system kernel. Each container appears to have its own file system, but the files are all stored in a common underlying file system across all containers. Processes within a container can interact with each other through the file system as well as interprocess communication, but they can interact with processes from other containers only via network connections.

Figure 20.11 depicts the different deployment alternatives for a set of applications. Figure 20.11a shows the alternative of multiple applications running in a single machine, sharing libraries and operating-system kernel. Figure 20.11b shows the alternative of running each application in its own VM, with multiple VMs running on a single machine. The different VMs running on a single real machine are managed by a software layer called the *hypervisor*. Figure 20.11c shows the alternative of using containers, with each container having its own libraries, and multiple containers running on a single machine. Since containers have lower overheads, a single machine can support more containers than VMs.

Containers provide low-cost support for elasticity, since more containers can be deployed very quickly on existing virtual machines, instead of starting up fresh virtual machines.

Many applications today are built as a collection of multiple **services**, each of which runs as a separate process, offering a network API; that is, the functions provided by the service are invoked by creating a network connection to the process and sending a service request over the network connection. Such an application architecture, which builds an application as a collection of small services, is called a **microservices architecture**. Containers fit the microservices architecture very well, since they provide a very low overhead mechanism to execute processes supporting each service.

We can simulate this model easily by using the other architectures, since transfer of data can be done via shared memory in a shared-memory architecture, and via shared disks in a shared-disk architecture. Hence, algorithms for shared-nothing architectures can be used on the other architectures too. In Section 22.6, we discuss how some of the algorithms can be further optimized for shared-memory systems.

Current-generation parallel systems are typically based on a hybrid architecture, where each computer has multiple cores with a shared memory, and there are multiple computers organized in a shared-nothing fashion. For the purpose of our discussion, with such an architecture, each core can be considered a node in a shared-nothing system. Optimizations to exploit the fact that some of the cores share memory with other cores can be performed as discussed in Section 22.6.

22.2 Parallel Sort

Suppose that we wish to sort a relation r that resides on n nodes $N_1, N_2, \dots, N_n$. If the relation has been range-partitioned on the attributes on which it is to be sorted, we can sort each partition separately and concatenate the results to get the full sorted relation. Since the tuples are partitioned on n nodes, the time required for reading the entire relation is reduced by a factor of n by the parallel access.

If relation r has been partitioned in any other way, we can sort it in one of two ways:

1. We can range-partition r on the sort attributes, and then sort each partition separately.
2. We can use a parallel version of the external sort-merge algorithm.

22.2.1 Range-Partitioning Sort

Range-partitioning sort, shown pictorially in Figure 22.1a, works in two steps: first range-partitioning the relation, then sorting each partition separately. When we sort by range-partitioning the relation, it is not necessary to range-partition the relation on the same set of nodes as those on which that relation is stored. Suppose that we choose nodes $N_1, N_2, \dots, N_m$ to sort the relation. There are two steps involved in this operation:

1. Redistribute the tuples in the relation, using a range-partition strategy, so that all tuples that lie within the i th range are sent to node n_i , which stores the relation temporarily on its local disk.

To implement range partitioning, in parallel every node reads the tuples from its disk and sends each tuple to its destination node based on the partition function. Each node $N_1, N_2, \dots, N_m$ also receives tuples belonging to its partition and stores them locally. This step requires disk I/O and network communication.

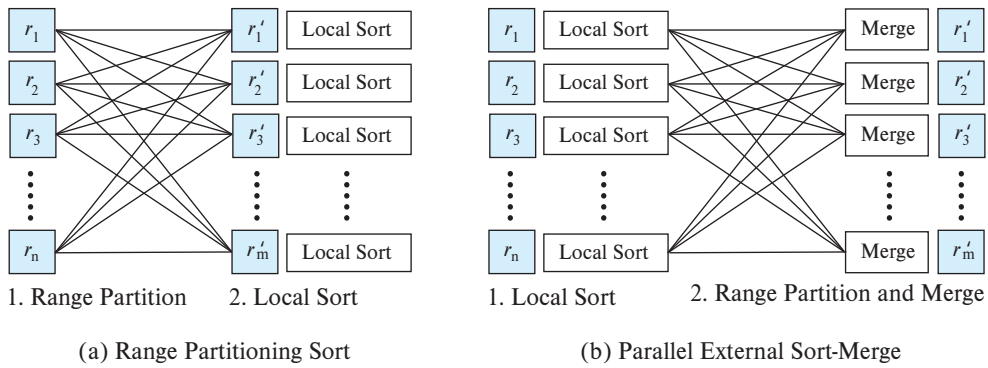


Figure 22.1 Parallel sorting algorithms.

- Each of the nodes sorts its partition of the relation locally, without interaction with the other nodes. Each node executes the same operation—namely, sorting—on a different data set. (Execution of the same operation in parallel on different sets of data are called **data parallelism**.)

The final merge operation is trivial, because the range partitioning in the first phase ensures that, for $1 \leq i < j \leq m$, the key values in node N_i are all less than the key values in N_j .

We must do range partitioning with a balanced range-partition vector so that each partition will have approximately the same number of tuples. We saw how to create such partition vectors in Section 21.3.1. Virtual node partitioning, as discussed in Section 21.3.2, can also be used to reduce skew. Recall that there are several times as many virtual nodes as real nodes, and virtual node partitioning creates a partition for each virtual node. Virtual nodes are then mapped to real nodes; doing so in a round-robin fashion tends to spread virtual nodes across real nodes in a way that reduces the degree of skew at real nodes.

22.2.2 Parallel External Sort-Merge

Parallel external sort-merge, shown pictorially in Figure 22.1b, is an alternative to range partitioning sort. Suppose that a relation has already been partitioned among nodes $N_1, N_2, \dots, N_n$ (it does not matter how the relation has been partitioned). Parallel external sort-merge then works this way:

- Each node N_i sorts the data available at N_i .
- The system then merges the sorted runs on each node to get the final sorted output.

The merging of the sorted runs in step 2 can be parallelized by this sequence of actions:

1. The system range-partitions the sorted partitions at each node N_i (all by the same partition vector) across the nodes $N_1, N_2, \dots, N_m$. It sends the tuples in sorted order, so each node receives the tuples as sorted streams.
2. Each node N_i performs a merge on the streams of tuples as they are received to get a single sorted run.
3. The system concatenates the sorted runs on nodes $N_1, N_2, \dots, N_m$ to get the final result.

As described, this sequence of actions results in an interesting form of **execution skew**, since at first every node sends all tuples of partition 1 to N_1 , then every node sends all tuples of partition 2 to N_2 , and so on. Thus, while sending happens in parallel, receiving tuples becomes sequential: First only N_1 receives tuples, then only N_2 receives tuples, and so on. To avoid this problem, the sorted sequence of tuples $S_{i,j}$ from any node i destined to any other node j is broken up into multiple blocks. Each node N_i sends the first block of tuples from $S_{i,j}$ node N_j , for each j ; it then sends the second block of tuples to each node N_j , and so on, until all blocks have been sent. As a result, all nodes receive data in parallel. (Note that tuples are sent in blocks, rather than individually, to reduce network overheads.)

22.3 Parallel Join

Parallel join algorithms attempt to divide the tuples of the input relations over several nodes. Each node then computes part of the join locally. Then, the system collects the results from each node to produce the final result. How exactly to divide the tuples depends on the join algorithm, as we see next.

22.3.1 Partitioned Join

For certain kinds of joins, it is possible to *partition* the two input relations across the nodes and to compute the join locally at each node. The partitioned join technique can be used for inner joins, where the join condition is an equi-join (e.g., $r \bowtie_{r.A=s.B} s$); the relations r and s are partitioned by the same partitioning function on their join attributes. The idea of partitioning is exactly the same as that behind the partitioning step of hash join. Partitioned join can also be used for outer joins, as we shall see shortly.

Suppose that we are using m nodes to perform the join, and that the relations to be joined are r and s . **Partitioned join** then works this way: The system partitions the relations r and s each into m partitions, denoted $r_1, r_2, \dots, r_m$ and $s_1, s_2, \dots, s_m$. In a partitioned join, however, there are two different ways of partitioning r and s :

- Range partitioning on the join attributes.
- Hash partitioning on the join attributes.

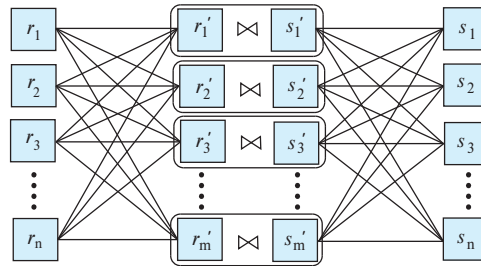
In either case, the same partitioning function must be used for both relations. For range partitioning, the same partition vector must be used for both relations. For hash partitioning, the same hash function must be used on both relations. Figure 22.2 depicts the partitioning in a partitioned parallel join.

The partitioned join algorithm first partitions one of the relations by scanning its tuples and sending them to the appropriate node based on the partition function and the join attribute values of each tuple. Specifically, each node N_i reads in the tuples of one of the relations, say r , from local disk, computes for each tuple t the partition r_j to which t belongs, and sends the tuple t to node N_j . Each node also simultaneously receives tuples that are sent to it and stores them on its local disk (this can be done by having separate threads for sending and receiving data). The process is repeated for all tuples from the other relation, s .

Once both relations are partitioned, we can use any join technique locally at each node N_i to compute the join of r_i and s_i . Thus, we can use partitioning to parallelize any join technique.

Partitioned join can be used not only for inner joins, but also for all three forms of outer join (left, right and full outer join). Each node computes the corresponding outer join locally, after partitioning is done on the join attributes. Further, since natural join can be expressed as an equijoin followed by a projection, natural joins can also be computed using partitioned join.

If one or both of the relations r and s are already partitioned on the join attributes (by either hash partitioning or range partitioning), the work needed for partitioning is reduced greatly. If the relations are not partitioned or are partitioned on attributes other than the join attributes, then the tuples need to be repartitioned.



Step 1: Partition r Step 2: Partition s
 Step 3: Each node N_i computes $r_i' \bowtie s_i'$

Figure 22.2 Partitioned parallel join.

We now consider issues specific to the join technique used locally at each node N_i . The local join operation can be optimized by performing some initial steps on tuples as they arrive at a node, instead of first storing the tuples to disk and then reading them back to perform these initial steps. These optimizations, which we describe below, are also used in nonparallel query processing, when results of an earlier operation are pipelined into a subsequent operation; thus, they are not specific to parallel query processing.

- If we use hash join locally, the resultant parallel join technique is called **partitioned parallel hash join**.

Recall that hash join first partitions both input relations into smaller pieces such that each partition of the smaller relation (the build relation) fits into memory. Thus, to implement hash join, the partitions r_i and s_i received by node N_i must be repartitioned using a hash function, say $h1()$. If the partitioning of r and s across the nodes was done by using a hash function $h0()$, the system must ensure that $h1()$ is different from $h0()$. Let the resultant partitions at node N_i be $r_{i,j}$ and $s_{i,j}$ for $j = 1 \dots n_i$, where n_i denotes the number of local partitions at node N_i .

Note that the tuples can be repartitioned based on the hash function used for the local hash join as they arrive and written out to the appropriate partitions, avoiding the need to write the tuples to disk and read them back in.

Recall also that hash join then loads each partition of the build relation into memory, builds an in-memory index on the join attributes, and finally probes the in-memory index using each tuple of the other relation, called the probe relation. Assume that relation s is chosen as the build relation. Then each partition $s_{i,j}$ is loaded in memory, with an index built on the join attributes, and the index is probed with each tuple of $r_{i,j}$.

Hybrid hash join (described in Section 15.5.5.5) can be used in case the partitions of one of the relations are small enough that a significant part of the partition fits in memory at each node. In this case, the smaller relation, say s , which is used as the build relation, should be partitioned first, followed by the larger relation, say r , which is used as the probe relation. Recall that with hybrid hash join, the tuples in the partition s_0 of the build relation s are retained in memory, and an in-memory index is built on these tuples. When the probe relation tuples arrive at the node, they are also repartitioned; tuples in the r_0 partition are used directly to probe the index on the s_0 tuples, instead of being written out to disk and read back in.

- If we use merge join locally, the resultant technique is called **partitioned parallel merge join**. Each of the partitions s_i and r_i must be sorted, and merged locally, at node N_i .

The first step of sorting, namely, run generation, can directly consume incoming tuples to generate runs, avoiding a write to disk before run generation.

- If we use nested-loops or indexed nested-loops join locally, the resultant technique is called **partitioned parallel nested-loop join** or **partitioned parallel indexed nested-**

loops join. Each node N_i performs a nested-loops (or indexed nested-loops) join on s_i and r_i .

22.3.2 Fragment-and-Replicate Join

Partitioning is not applicable to all types of joins. For instance, if the join condition is an inequality, such as $r \bowtie_{r.a < s.b} s$, it is possible that all tuples in r join with some tuple in s (and vice versa). Thus, there may be no nontrivial way of partitioning r and s so that tuples in partition r_i join with only tuples in partition s_i .

We can parallelize such joins by using a technique called *fragment-and-replicate*. We first consider a special case of fragment-and-replicate—**asymmetric fragment-and-replicate join**—which works as follows:

1. The system partitions one of the relations—say, r . Any partitioning technique can be used on r , including round-robin partitioning.
2. The system replicates the other relation, s , across all the nodes.
3. Node N_i then locally computes the join of r_i with all of s , using any join technique.

The asymmetric fragment-and-replicate scheme appears in Figure 22.3a. If r is already stored by partitioning, there is no need to partition it further in step 1. All that is required is to replicate s across all nodes.

The asymmetric fragment-and-replicate join technique is also referred to as **broadcast join**. It is a very useful technique, even for equi-joins, if one of the relations, say s , is small, and the other relation, say r , is large, since replicating the small relation s across all nodes may be cheaper than repartitioning the large relation r .

The general case of **fragment-and-replicate join** (also called the **symmetric fragment-and-replicate join**) appears in Figure 22.3b; it works this way: The system partitions relation r into n partitions, $r_1, r_2, \dots, r_n$, and partitions s into m partitions, $s_1, s_2, \dots, s_m$. As before, any partitioning technique may be used on r and on s . The values of m and n do not need to be equal, but they must be chosen so that there are at least $m * n$ nodes. Asymmetric fragment-and-replicate is simply a special case of general fragment-and-replicate, where $m = 1$. Fragment-and-replicate reduces the sizes of the relations at each node, compared to asymmetric fragment-and-replicate.

Let the nodes be $N_{1,1}, N_{1,2}, \dots, N_{1,m}, N_{2,1}, \dots, N_{n,m}$. Node $N_{i,j}$ computes the join of r_i with s_j . To ensure that each node $N_{i,j}$ gets all tuples of r_i and s_j , the system replicates r_i to nodes $N_{i,1}, N_{i,2}, \dots, N_{i,m}$ (which form a row in Figure 22.3b), and replicates s_j to nodes $N_{1,j}, N_{2,j}, \dots, N_{n,j}$ (which form a column in Figure 22.3b). Any join technique can be used at each node $N_{i,j}$.

Fragment-and-replicate works with any join condition, since every tuple in r can be tested with every tuple in s . Thus, it can be used where partitioning cannot be used. However, note that each tuple in r is replicated m times, and each tuple in s is replicated n times.

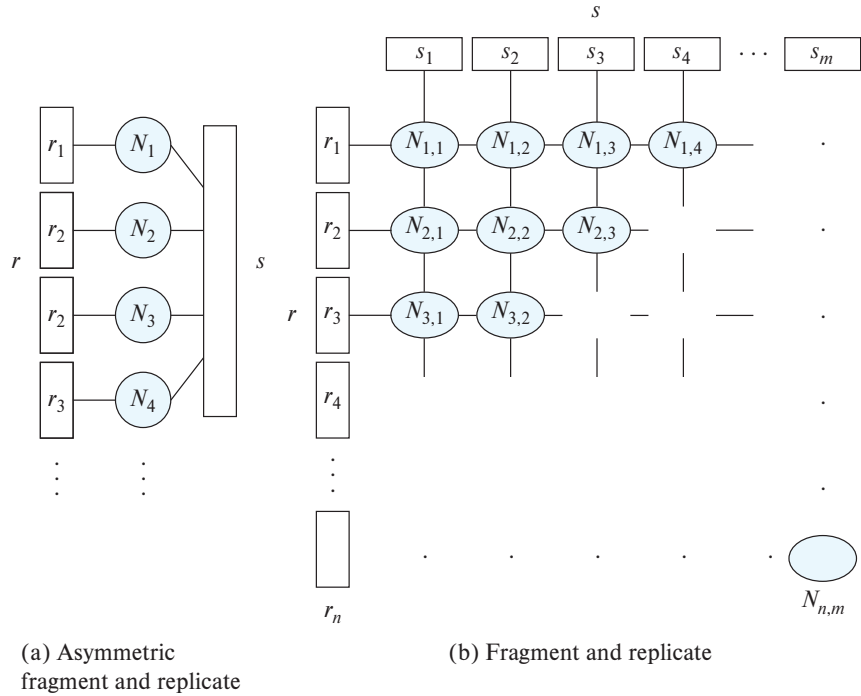


Figure 22.3 Fragment-and-replicate schemes.

Fragment-and-replicate join has a higher cost than partitioning, since it involves replication of both relations, and is therefore used only if the join does not involve equi-join conditions. Asymmetric fragment-and-replicate, on the other hand, is useful even for equi-join conditions, if one of the relations is small, as discussed earlier.

Note that asymmetric fragment-and-replicate join can be used to compute the left outer join operation $r \bowtie_{\theta} s$ if s is replicated, by simply computing the left outer join locally at each node. There is no restriction on the join condition θ .

However, $r \bowtie_{\theta} s$ cannot be computed locally if s is fragmented and r is replicated, since an r tuple may have no matching tuple in partition s_i , but may have a matching tuple in partition $s_j, j \neq i$. Thus, a decision on whether or not to output the r tuple with null values for s attributes cannot be made locally at node N_i . For the same reason, asymmetric fragment-and-replicate cannot be used to compute the full outer join operation, and symmetric fragment-and-replicate cannot be used to compute any of the outer join operations.

22.3.3 Handling Skew in Parallel Joins

Skew presents a special problem for parallel join techniques. If one of the nodes has a much heavier load than other nodes, the parallel join operation will take much longer to finish, with many idle nodes waiting for the heavily loaded node to finish its task.

When partitioning data for storage, to minimize skew in storage we use a balanced partitioning vector that ensures all nodes get the same number of tuples. For parallel joins, we need to instead balance the execution time of join operations across all nodes. Hash partitioning using any good hash function usually works quite well at balancing the load across nodes, unless some join attribute values occur very frequently. Range partitioning, on the other hand, is more vulnerable to join skew, unless the ranges are carefully chosen to balance the load.¹

Virtual-node partitioning with, say, round-robin distribution of virtual nodes to real nodes, can help in reducing skew at the level of real nodes even if there is skew at the level of virtual nodes, since the skewed virtual nodes tend to get spread over multiple real nodes.

The preceding techniques are examples of **join skew avoidance**. Virtual-node partitioning, in particular, is very effective at skew avoidance in most cases.

However, there are cases with high skew, for example where some join attribute values are very frequent in both input relations, leading to a large join result size. In such cases, there could be significant join skew, even with virtual-node partitioning.

Dynamic handling of join skew is an alternative to skew avoidance. A dynamic approach can be used to detect and handle skew in such situations. Virtual node partitioning is used, and the system then monitors the join progress at each real node. Each real node schedules one virtual node at a time. Suppose that some real node has completed join processing for all virtual nodes assigned to it, and is thus idle, while some other real node has multiple virtual nodes waiting to be processed. Then, the idle node can get a copy of the data corresponding to one of the virtual nodes at the busy node and process the join for that virtual node. This process can be repeated whenever there is an idle real node, as long as some real node has virtual nodes waiting to be processed.

This technique is an example of **work stealing**, where a processor that is idle takes work that is in the queue of another processor that is busy. Work stealing is inexpensive in a shared-memory system, since all data can be accessed quickly from the shared memory, as discussed further in Section 22.6. In a shared-nothing environment, data movement may be required to move a task from one processor to another, but it is often worth paying the overhead to reduce the completion time of a task.

22.4 Other Operations

In this section, we discuss parallel processing of other relational operations, as well as parallel processing in the MapReduce framework.

¹Cost estimation should be done using histograms on join attributes. A heuristic approximation is to estimate the join cost at each node N_i as the sum of the sizes of r_i and s_i , and choose range partitioning vectors to balance the sum of the sizes.

22.4.1 Other Relational Operations

The evaluation of other relational operations also can be parallelized:

- **Selection.** Let the selection be $\sigma_\theta(r)$. Consider first the case where θ is of the form $a_i = v$, where a_i is an attribute and v is a value. If the relation r is partitioned on a_i , the selection proceeds at a single node. If θ is of the form $l \leq a_i \leq u$ —that is, θ is a range selection—and the relation has been range-partitioned on a_i , then the selection proceeds at each node whose partition overlaps with the specified range of values. In all other cases, the selection proceeds in parallel at all the nodes.
- **Duplicate elimination.** Duplicates can be eliminated by sorting; either of the parallel sort techniques can be used, optimized to eliminate duplicates as soon as they appear during sorting. We can also parallelize duplicate elimination by partitioning the tuples (by either range or hash partitioning) and eliminating duplicates locally at each node.
- **Projection.** Projection without duplicate elimination can be performed as tuples are read in from disk in parallel. If duplicates are to be eliminated, either of the techniques just described can be used.
- **Aggregation.** Consider an aggregation operation. We can parallelize the operation by partitioning the relation on the grouping attributes, and then computing the aggregate values locally at each node. Either hash partitioning or range partitioning can be used. If the relation is already partitioned on the grouping attributes, the first step can be skipped.

We can reduce the cost of transferring tuples during partitioning by partly computing aggregate values before partitioning, at least for the commonly used aggregate functions. Consider an aggregation operation on a relation r , using the **sum** aggregate function on attribute B , with grouping on attribute A . The system can perform the **sum** aggregation at each node N_i on those r tuples stored at N_i . This computation results in tuples with partial sums at each node; the result at N_i has one tuple for each A value present in r tuples stored at N_i , with the sum of the B values of those tuples. The system then partitions the result of the local aggregation on the grouping attribute A and performs the aggregation again (on tuples with the partial sums) at each node N_i to get the final result.

As a result of this optimization, which is called **partial aggregation**, fewer tuples need to be sent to other nodes during partitioning. This idea can be extended easily to the **min** and **max** aggregate functions. Extensions to the **count** and **avg** aggregate functions are left for you to do in Exercise 22.2.

Skew handling for aggregation is easier than skew handling for joins, since the cost of aggregation is directly proportional to the input size. Usually, all that needs to be done is to use a good hash function to ensure the group-by attribute values are evenly distributed amongst the participating nodes. However, in some extreme cases, a few

values occur very frequently in the group-by attributes, and hashing can lead to uneven distribution of values. When applicable, partial aggregation is very effective in avoiding skew in such situations. However, when partial aggregation is not applicable, skew can occur with aggregation.

Dynamic detection and handling of such skew can be done in some such cases: in case a node is found to be overloaded, some of the key values that are not yet processed by the node can be reassigned to another node, to balance the load. Such reassignment is greatly simplified if virtual-node partitioning is used; in that case, if a real node is found to be overloaded, some virtual nodes assigned to the overloaded real node, but not yet processed, are identified, and reassigned to other real nodes.

More information on skew handling for join and other operators may be found in the Further Reading section at the end of the chapter.

22.4.2 Map and Reduce Operations

Recall the MapReduce paradigm, described in Section 10.3, which is designed to ease the writing of parallel data processing programs.

Recall that the `map()` function provided by the programmer is invoked on each input record and emits zero or more output data items, which are then passed on to the `reduce()` function. Each data item output by a `map()` function consists of a record (*key*, *value*); we shall call the key as the **intermediate key**. In general, a `map()` function can emit multiple such records and since there are many input records, there are potentially many output records overall.

The MapReduce system takes all the records emitted by the `map()` functions, and groups them such that all records with a particular intermediate key are gathered together. The `reduce()` function provided by the programmer is then invoked for each intermediate key and iterates over a collection of all values associated with that key.

Note that the `map` function can be thought of as a generalization of the project operation: both process a single record at a time, but for a given input record the project operation generates a single output record, whereas the `map` function can output multiple records (including, as a special case, 0 records). Unlike the project operation, the output of a `map` function is usually intended to become the input of a `reduce` function; hence, the output of a `map` function has an associated key that serves as a group by attribute. Recall that the `reduce` function takes as input a collection of values and outputs a result; with most of the `reduce` functions commonly in use, the result is an aggregate computed on the input values, and the `reduce` function is then essentially a user-defined aggregation function.

MapReduce systems are designed for parallel processing of data. A key requirement for parallel processing is the ability to parallelize file input and output across multiple machines; otherwise, the single machine storing the data will become a bottleneck. Parallelization of file input and output can be done by using a distributed file system, such as the *Hadoop File System (HDFS)*, discussed in Section 21.6, or by using a parallel/distributed storage system, discussed in Section 21.7. Recall that in such sys-

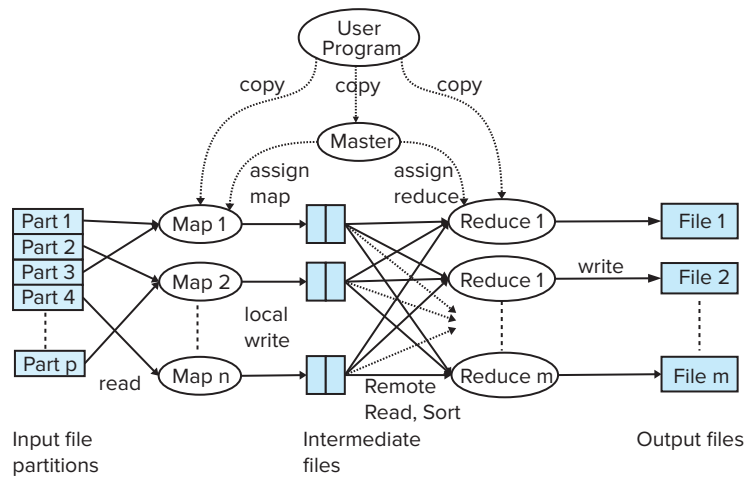


Figure 22.4 Parallel processing of MapReduce job.

tems, data are replicated (copied) across several (typically 3) machines, so that even if a few of the machines fail, the data are available from other machines that have copies of the data in the failed machine.

Conceptually, the map and reduce operations are parallelized in the same way that the relational operations project and aggregation are parallelized. Each node in the system has a number of concurrently executing **workers**, which are processes that execute map and reduce functions. The number of workers on one machine is often set to match the number of processor cores on the machine.

Parallel processing of MapReduce jobs is shown schematically in Figure 22.4. As shown in the figure, MapReduce systems split the input data into multiple pieces; the job of processing one such piece is called a **task**. Splitting can be done in units of files, and large files can be split into multiple parts. Tasks correspond to virtual nodes in our terminology, while workers correspond to real nodes. Note that with a multicore processor (as is standard today), MapReduce systems typically allocate one worker per core.

MapReduce systems also have a scheduler, which assigns tasks to workers.² Whenever a worker completes a task, it is assigned a new task, until all tasks have been assigned.

A key step between the map and reduce operations is the repartitioning of records output by the map step; these records are repartitioned based on their intermediate (reduce) key, such that all records with a particular key are assigned to the same reducer task. This could be done either by range-partitioning on the reduce key or by computing a hash function on the reduce key. In either case, the records are divided into multiple

²The scheduler is run on a dedicated node called the master node; the nodes that perform `map()` and `reduce()` tasks are called slave nodes in the Hadoop MapReduce terminology.

partitions, each of which is called a reduce task. A scheduler assigns reduce tasks to workers.

This step is identical to the repartitioning done for parallelizing the relational aggregation operation, with records partitioned into a number of virtual nodes based on their group-by key.

To process the records in a particular **reduce** task, the records are sorted (or grouped) by the reduce key, so that all records with the same reduce-key value are brought together, and then the `reduce()` is executed on each group of reduce-key values.

The reduce tasks are executed in parallel by the workers. When a worker completes a reduce task, another task is assigned to it, until all reduce tasks have been completed. A reduce task may have multiple different reduce key values, but a particular call to the `reduce()` function is for a single reduce key; thus, the `reduce()` function is called for each key in the reduce task.

Tasks correspond to virtual nodes in the virtual-node partitioning scheme. There are far more tasks than there are nodes, and tasks are divided among the nodes. As discussed in Section 22.3.3, virtual-node partitioning reduces skew. Also note that as discussed in Section 22.4.1, skew can be reduced by partial aggregation, which corresponds to *combiners* in the MapReduce framework.

Further, MapReduce implementations typically also carry out dynamic detection and handling of skew, as discussed in Section 22.4.1.

Most MapReduce implementations include techniques to ensure that processing can be continued even if some nodes fail during query execution. Details are discussed further in Section 22.5.4.

22.5 Parallel Evaluation of Query Plans

As discussed in Section 22.1, there are two types of parallelism: intraoperation and interoperation. Until now in this chapter, we have focused on intraoperation parallelism. In this section, we consider execution plans for queries containing multiple operations.

We first consider how to exploit interoperator parallelism. We then consider a model of parallel query execution which breaks parallel query processing into two types of steps: partitioning of data using the *exchange operator*, and execution of operations on purely local data, without any data exchange. This model is surprisingly powerful and is widely used in parallel database implementations.

22.5.1 Interoperation Parallelism

There are two forms of interoperation parallelism: pipelined parallelism and independent parallelism. We first describe these forms of parallelism, assuming each operator runs on a single node without intraoperation parallelism.

We then describe a model for parallel execution based on the exchange operator, in Section 22.5.2. Finally, in Section 22.5.3, we describe how a complete plan can be executed, combining all the forms of parallelism.

22.5.1.1 Pipelined Parallelism

Recall from Section 15.7.2 that in pipelining, the output tuples of one operation, A , are consumed by a second operation, B , even before the first operation has produced the entire set of tuples in its output. The major advantage of pipelined execution in a sequential evaluation is that we can carry out a sequence of such operations without writing any of the intermediate results to disk.

Parallel systems use pipelining primarily for the same reason that sequential systems do. However, pipelines are a source of parallelism as well, since it is possible to run operations A and B simultaneously on different nodes (or different cores of a single node), so that B consumes tuples in parallel with A producing them. This form of parallelism is called **pipelined parallelism**.

Pipelined parallelism is useful with a small number of nodes, but it does not scale up well. First, pipeline chains generally do not attain sufficient length to provide a high degree of parallelism. Second, it is not possible to pipeline relational operators that do not produce output until all inputs have been accessed, such as the set-difference operation. Third, only marginal speedup is obtained for the frequent cases in which one operator's execution cost is much higher than are those of the others.

All things considered, when the degree of parallelism is high, pipelining is a less important source of parallelism than partitioning. The real reason for using pipelining with parallel query processing is the same reason that pipelining is used with sequential query processing: namely, that pipelined executions can avoid writing intermediate results to disk.

Pipelining in centralized databases was discussed in Section 15.7.2; as mentioned there, pipelining can be done using a demand-driven, or pull, model of computation, or using a producer-driven, or push, model of computation. The pull model is widely used in centralized database systems.

However, the push model is greatly preferred in parallel database systems, since, unlike the pull model, the push model allows both the producer and consumer to execute in parallel.

Unlike the pull model, the push model requires a buffer that can hold multiple tuples, between the producer and consumer; without such a buffer, the producer would stall as soon as it generates one tuple. Figure 22.5 shows a producer and consumer with a buffer in-between. If the producer and consumer are on the same node, as shown in Figure 22.5a, the buffer can be in shared memory. However, if the producer and consumer are in different nodes, as shown in Figure 22.5b, there will be two buffers: one at the producer node to collect tuples as they are produced, and another at the consumer node to collect them as they are sent across the network.

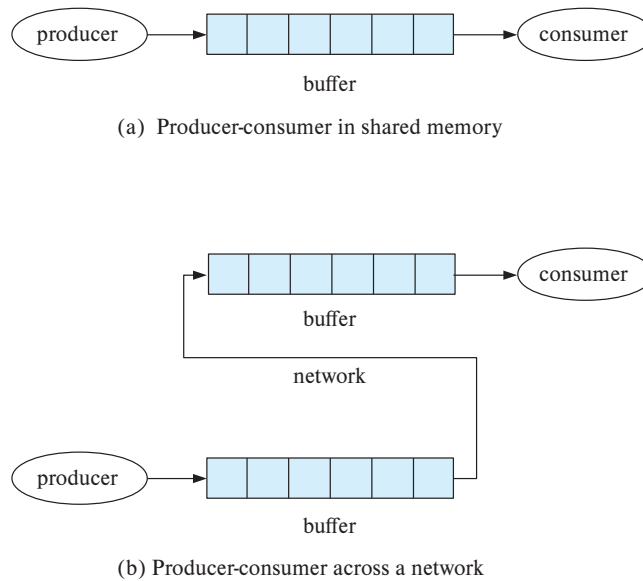


Figure 22.5 Producer and consumer with buffer.

When sending tuples across a network, it makes sense to collect multiple tuples and send them as a single *batch*, rather than send tuples one at a time, since there is usually a very significant overhead per message. Batching greatly reduces this overhead.

If the producer and consumer are on the same node and can communicate via a shared memory buffer, mutual exclusion needs to be ensured when inserting tuples into, or fetching tuples from, the buffer. Mutual exclusion protocols have some overhead, which can be reduced by inserting/retrieving a batch of tuples at a time, instead of one tuple at a time.

Note that with the pull model, either the producer or the consumer, but not both, can be executing at a given time; while this avoids the contention on the shared buffer that arises with the use of the push model, it also prevents the producer and consumer from running concurrently.

22.5.1.2 Independent Parallelism

Operations in a query expression that do not depend on one another can be executed in parallel. This form of parallelism is called **independent parallelism**.

Consider the join $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$. One possible plan is to compute intermediate result $t_1 \leftarrow r_1 \bowtie r_2$ in parallel with intermediate result $t_2 \leftarrow r_3 \bowtie r_4$. Neither of these computations depends on each other, and hence they can be parallelized by independent parallelism. In other words, the execution of these two joins can be scheduled in parallel.

When these two computations complete, we can compute:

$$t_1 \bowtie t_2$$

Note that computation of the above join depends on the results of the first two joins, hence it cannot be done using independent parallelism.

Like pipelined parallelism, independent parallelism does not provide a high degree of parallelism and is less useful in a highly parallel system, although it is useful with a lower degree of parallelism.

22.5.2 The Exchange Operator Model

The Volcano parallel database popularized a model of parallelization called the *exchange-operator* model. The exchange operation repartitions data in a specified way; data interchange between nodes is done only by the exchange operator. All other operations work on local data, just as they would in a centralized database system; the data may be available locally either because it is already present, or because of the execution of a preceding exchange operator.

The **exchange operator** has two components: a scheme for *partitioning* outgoing data, applied at each source node, and a scheme for *merging* incoming data, applied at each destination node. The operator is shown pictorially in Figure 22.6, with the partitioning scheme denoted as “Partition,” and the merging scheme denoted as “Merge.”

The exchange operator can *partition* data in one of several ways:

1. By hash partitioning on a specified set of attributes.
2. By range partitioning on a specified set of attributes.
3. By replicating the input data at all nodes, referred to as **broadcasting**.
4. By sending all data to a single node.

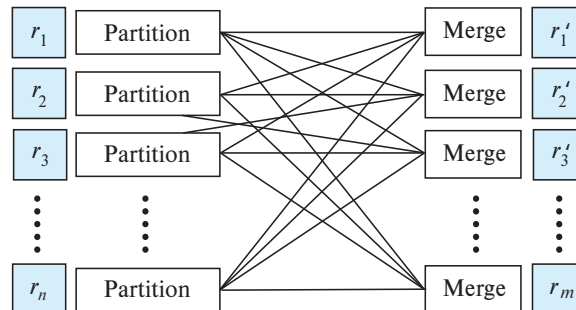


Figure 22.6 The exchange operator used for repartitioning.

Broadcasting data to all nodes is required for operations such as the asymmetric fragment-and-replicate join. Sending all data to a single node is usually done as a final step of parallel query processing, to get partitioned results together at a single site.

Note also that the input to the exchange operator can be at a single site (referred to as **unpartitioned**), or already partitioned across multiple sites. Repartitioning of already partitioned data results in each destination node receiving data from multiple source nodes, as shown in Figure 22.6.

Each destination node *merges* the data items received from the source nodes. This merge step can store data in the order received (which may be nondeterministic, since it depends on the speeds of the machines and unpredictable network delays); such merging is called **random merge**.

On the other hand, if the input data from each source is sorted, the merge step can exploit the sort order by performing an **ordered merge**. Suppose, for example, nodes $N_1, \dots, N_m$ first sort a relation locally, and then repartition the sorted relation using range partitioning. Each node performs an ordered merge operation on the tuples that it receives, to generate a sorted output locally.

Thus, the exchange operator performs the partitioning of data at the source nodes, as well the merging of data at the destination nodes.

All the parallel operator implementations we have seen so far can be modeled as a sequence of exchange operations, and local operators, at each node, that are completely unaware of parallelism.

- Range partitioning sort: can be implemented by an exchange operation that performs range partitioning, with random merge at the destination nodes, followed by a local sort operation at each destination node.
- Parallel external sort-merge: can be implemented by local sorting at the source nodes, followed by an exchange operation that performs range partitioning, along with ordered merging.
- Partitioned join: can be implemented by an exchange operation that performs the desired partitioning, followed by local join at each node.
- Asymmetric fragment-and-replicate join: can be implemented by an exchange operation that performs broadcast “partitioning” of the smaller relation, followed by a local join at each node.
- Symmetric fragment-and-replicate join: can be implemented by an exchange operation that partitions, and partially broadcasts each partition, followed by a local join at each node.
- Aggregation: can be implemented by an exchange operation that performs hash-partitioning on the grouping attributes, followed by a local aggregation operation at each node. The partial-aggregation optimization simply requires an extra local aggregation operation at each node, before the exchange operation.

Other relational operations can be implemented similarly, by a sequence of local operations running in parallel at each node, interspersed with exchange operations.

As noted earlier, parallel execution where data are partitioned, and operations are executed locally at each node, is referred to as *data parallelism*. The use of the exchange operator model to implement data parallel execution has the major benefit of allowing existing database query engines to be used at each of the local nodes, without any significant code changes. As a result, the exchange-operator model of parallel execution is widely used in parallel database systems.

There are, however, some operator implementations that can benefit from being aware of the parallel nature of the system they are running on. For example, an indexed nested-loops join where the inner relation is indexed on a parallel data-store would require remote access for each index lookup; the index lookup operation is thus aware of the parallel nature of the underlying system. Similarly, in a shared-memory system it may make sense to have a hash table or index in shared-memory, which is accessed by multiple processors (this approach is discussed briefly in Section 22.6); the operations running on each processor are then aware of the parallel nature of the system.

As we discussed in Section 22.5.1.1, while the demand-driven (or pull) iterator model for pipelined execution of operators is widely used in centralized database engines, the push model is preferred for parallel execution of operators in a pipeline.

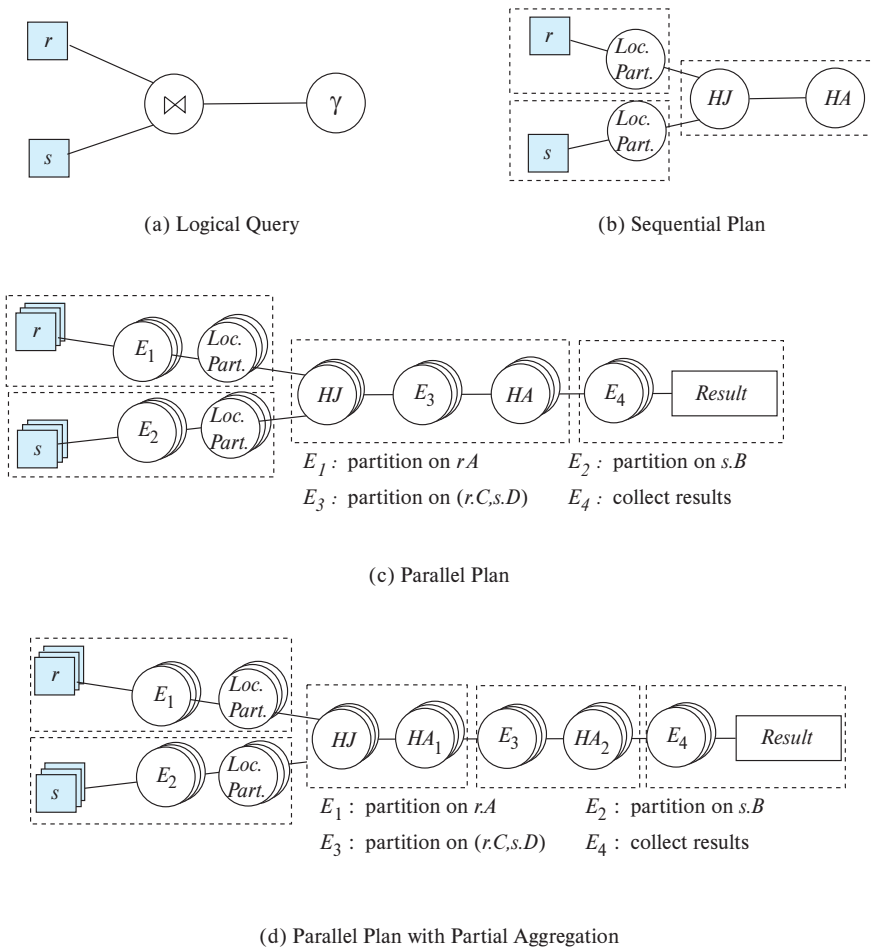
The exchange operator can be used to implement the push model between nodes in a parallel system, while allowing existing implementations of local operators to run using the pull model. To do so, at each source node of an exchange operator, the operator can pull multiple tuples from its input and create a batch of tuples destined for each destination node. The input may be computed by a local operation, whose implementation can use the demand-driven iterator model.

The exchange operator then sends batches of tuples to the destination nodes, where they are merged and kept in a buffer. The local operations can then consume the tuples in a demand-driven manner.

22.5.3 Putting It All Together

Figure 22.7 shows a query, along with a sequential and two alternative parallel query execution plans. The query, shown in Figure 22.7a, computes a join of two relations, r and s , and then computes an aggregate on the join result. Assume for concreteness that the query is $_{r.C,s.D}\gamma_{\text{sum}(s.E)}(r \bowtie_{r.A=s.B} s)$.

The sequential plan, shown in Figure 22.7b, uses a hash join (denoted as “HJ” in the figure), which executes in three separate stages. The first stage partitions the first input (r) locally on $r.A$; the second stage partitions the second input (s) locally on $s.B$; and the third stage computes the join of each of the corresponding partitions of r and s . The aggregate is computed using in-memory hash-aggregation, denoted by the operator HA; we assume that the number of groups is small enough that the hash table fits in memory.

**Figure 22.7** Parallel query execution plans.

The dashed boxes in the figure show which steps run in a pipelined fashion. In the sequential plan, the read of the relation r is pipelined to the first partitioning stage of the sequential hash join; similarly, the read of relations s is pipelined to the second partitioning stage of the hash join. The third stage of the hash join pipelines its output tuples to the hash aggregation operator.

The parallel query evaluation plan, shown in Figure 22.7c, starts with r and s already partitioned, but not on the required join attributes.³ The plan, therefore, uses the exchange operation E_1 to repartition r using attribute $r.A$; similarly, exchange operator E_2 repartitions s using $s.B$. Each node then uses hash join locally to compute the join

³Note the multiple boxes indicating a relation is stored in multiple nodes; similarly, multiple circles indicate that an operation is executed in parallel on multiple nodes.

of its partition of r and s . Note that these partitions are not assumed to fit in memory, so they must be further partitioned by the first two stages of hash join; this local partitioning step is denoted as “Loc. Part.” in the figure. The dashed boxes indicate that the output of the exchange operator can be pipelined to the local partitioning step. As in the sequential plan, there are two pipelined stages, one each for r and s . Note that exchange of tuples across nodes is done only by the exchange operator, and all other edges denote tuple flows within each node.

Subsequently, the hash join algorithm is executed in parallel at all participating nodes, and its output pipelined to the exchange operator E_3 . This exchange operator repartitions its input on the pair of attributes $(r.C, s.D)$, which are the grouping attributes of the subsequent aggregation. At the receiving end of the exchange operator, tuples are pipelined to the hash aggregation operator. Note that the above steps all run together as a single pipelined stage, even though there is an exchange operator as part of the stage. Note that the local operators computing hash join and hash aggregate need not be aware of the parallel execution.

The results of the aggregates are then collected together at a central location by the final exchange operator E_4 , to create the final result relation.

Figure 22.7d shows an alternative plan that performs partial aggregation on the results of the hash join, before partitioning the results. The partial aggregation is computed locally at each node by the operator HA_1 . Since no tuple is output by the partial aggregation operator until all its input is consumed, the pipelined stage contains only the local hash join and hash aggregation operators. The subsequent exchange operator E_3 which partitions its input on $(r.C, s.D)$ is part of a subsequent pipelined stage along with the hash aggregation operation HA_2 which computes the final aggregate values. As before, the exchange operator E_4 collects the results at a centralized location.

The above example shows how pipelined execution can be performed across nodes, as well as within nodes, and further how it can be done along with intra-operator parallel execution. The example also shows that some pipelined stages depend on the output of earlier pipelined stages; therefore their execution can start only after the previous step finishes. On the other hand, the initial exchange and partitioning of r and s occur in pipelined stages that are independent of each other; such independent stages can be scheduled concurrently, that is, at the same time, if desired.

To execute a parallel plan such as the one in our example, the different pipelined stages have to be scheduled for execution, in an order that ensures inter-stage dependencies are met. When executing a particular stage, the system must decide how many nodes an operation should be executed on. These decisions are usually made as part of the scheduling phase, before query execution starts.

22.5.4 Fault Tolerance in Query Plans

Parallel processing of queries across a moderate number of nodes, for example, hundreds of nodes, can be done without worrying about fault tolerance. If a fault occurs, the query is rerun, after removing any failed nodes from the system (replication of data

at the storage layer ensures that data continues to be available even in the event of a failure). However, this simple solution does not work well when operating at the scale of thousands or tens of thousands of nodes: if a query runs for several hours, there is a significant chance that there will be a failure while the query is being executed. If the query is restarted, there is a significant chance of another failure while it is executing, which is obviously an undesirable situation.

To deal with this problem, the query processing system should ideally just be able to redo the actions of a failed node, without redoing the rest of the computation.

Implementations of MapReduce that are designed to work at a massively parallel scale can be made fault tolerant as follows:

1. Each map operation executed at each node writes its output to local files.
2. The next operation, which is a reduce operation, executes at each node; the operation execution at a node reads data from the files stored at multiple nodes, collects the data, and starts processing the data only after it has got all its required data.
3. The reduce operation writes its output to a distributed file system (or distributed storage system) that replicates data, so that the data would be available even in the event of a failure.

Let us now examine the reason why things are done as above. First, if a particular map node fails, the work done at that node can be redone at a backup node; the work done at other map nodes is not affected. Work is not carried out by reduce nodes until all the required data has been fetched; the failure of a map node just means the reduce nodes fetch data from the backup map nodes. There is certainly a delay while the backup node does its work, but there is no need to repeat the entire computation.

Further, once a reduce node has finished its work, its output goes to replicated storage to ensure it is not lost even if a data storage node fails. This means that if a reduce node fails before it completes its work, it will have to be reexecuted at a backup node; other reduce nodes are not affected. Once a reduce node has finished its work, there is no need to reexecute it.

Note that it is possible to store the output of a map node in a replicated storage system. However, this increases the execution cost significantly, and hence map output is stored in local storage, even at the risk of having to reexecute the work done by a map node in case it fails before all the reduce nodes have fetched the data that they require from that map node.

It is also worth noting that sometimes nodes do not completely fail, but run very slowly; such nodes are called *straggler* nodes. Even a single straggler node can delay all the nodes in the next step (if there is a following step), or delay task completion (if it is in the last step). Straggler nodes can be dealt with by treating them similar to failed nodes, and reexecuting their tasks on other nodes (the original task on the straggler node can also be allowed to continue, in case it finishes first). Such reexecution to deal with

consumers. In such systems, one copy of an operator is treated as a primary copy, and the other copy as a hot-spare replica (recall that we discussed hot-spares in Section 19.7).

What we have described above is a high-level view of how streaming data systems implement fault tolerance. References to more information on how to implement fault tolerance in streaming systems may be found in the Further Reading section at the end of the chapter.

22.9 Distributed Query Processing

The need for distributed query processing originally arose when organizations needed to execute queries across multiple databases that were often geographically distributed. However, today the same need arises because organizations have data stored in multiple different databases and data storage systems, and they need to execute queries that access multiples of these databases and data storage systems.

22.9.1 Data Integration from Multiple Data Sources

Different parts of an enterprise may use different databases, either because of a legacy of how they were automated, or because of mergers of companies. Migrating an entire organization to a common system may be an expensive and time-consuming operation. An alternative is to keep data in individual databases, but to provide users with a logical view of integrated data. The local database systems may employ different logical models, different data-definition and data-manipulation languages, and may differ in their concurrency-control and transaction-management mechanisms. Some of the sources of data may not be full-fledged database systems but may instead be data storage systems, or even just files in a file system. Yet another possibility is that the data source may be on the cloud and accessed as a web service. Queries may need access to data stored across multiple databases and data sources.

Manipulation of information located in multiple databases and other data sources requires an additional software layer on top of existing database systems. This layer creates the illusion of logical database integration without requiring physical database integration and is sometimes called a **federated database system**.

Database integration can be done in several different ways:

- The **federated database** approach creates a common schema, called a **global schema**, for data from all the databases/data sources; each database has its own **local schema**. The task of creating a unified global schema from multiple local schemas is referred to as **schema integration**.

Users can issue queries against the global schema. A query on a global schema must be translated into queries on the local schemas at each of the sites where the query has to be executed. The query results have to be translated back into the global schema and combined to get the final result.

In general, updates to the common schema also need to be mapped to updates to the individual databases; systems that support a common schema and queries, but not updates, against the schema are sometimes referred to as **mediator** systems.

- The **data virtualization** approach allows applications to access data from multiple databases/data sources, but it does not try to enforce a common schema. Users have to be aware of the different schemas used in different databases, but they do not need to worry about which data are stored on which database system, or about how to combine information from multiple databases.
- The **external data** approach allows database administrators to provide schema information about data that are stored in other databases, along with other information, such as connection and authorization information needed to access the data. Data stored in external sources that can be accessed from a database are referred to as **external data**. **Foreign tables** are views defined in a database whose actual data are stored in an external data source. Such tables can be read as well as updated, depending on what operations the external data source supports. Updates on foreign tables, if supported, must be translated into updates on the external data source.

Unlike the earlier-mentioned approaches, the goal here is not to create a full-fledged distributed database, but merely to facilitate access to data from other data sources. The SQL Management of External Data (SQL MED) component of the SQL standard defines standards for accessing external data sources from a database.

If a data source is a database that supports SQL, its data can be easily accessed using ODBC or JDBC connections. Data in parallel data storage systems that do not support SQL, such as HBase, can be accessed using the API methods that they provide.

A **wrapper** provides a view of data stored at a data source, in a desired schema. For example, if the system has a global schema, and the local database schema is different from the global schema, a wrapper can provide a view of the data in the global schema. Wrappers can even be used to provide a relational view of nonrelational data sources, such as web services, flat files (e.g., web logs), and directory systems.

Wrappers can also translate queries on the global schema into queries on the local schema, and translate results back into the global schema. Wrappers may be provided by individual sites, they may be written as part of the federated database system. Many relational databases today support wrappers that provide a relational view of data stored in file systems; such wrappers are specific to the type of data stored in the files.

If the goal of data integration is solely to run decision support queries, *data warehouses*, which we saw in Section 11.2, are a widely used alternative to database integration. Data warehouses import data from multiple sources into a single (possibly parallel) system, with a centralized schema. Data are typically imported periodically,

for example, once in a day or once in a few hours, although continuous import is also increasingly used. Raw data imported from the data sources are typically processed and cleaned before being stored in the data warehouse.

However, with the scale at which data are generated by some applications, creating and maintaining such a warehouse can be expensive. Furthermore, queries cannot access the most recent data, since there is a delay between updates on the source database and import of the updates to the data warehouse. On the other hand, query processing can be done more efficiently in a data warehouse; further, queries at a data warehouse do not affect the performance of other queries and transactions at the data source. Whether to use a data warehouse architecture, or to directly access data from the data sources in response to individual queries, is a decision that each enterprise has to make based on its needs.

The term **data lake** is used to refer to an architecture where data are stored in multiple data storage systems and in different formats, including in file systems, but can be queried from a single system. Data lakes are viewed as an alternative to data warehouses, since they do not require up-front effort to preprocess data, although they do require more effort when creating queries.

22.9.2 Schema and Data Integration

The first task in providing a unified view of data lies in creating a unified conceptual schema, a task that is referred to as **schema integration**. Each local system provides its own conceptual schema. The database system must integrate these separate schemas into one common schema. Schema integration is a complicated task, mainly because of the semantic heterogeneity. The same attribute names may appear in different local databases but with different meanings.

Schema integration requires the creation of a **global schema**, which provides a unified view of data in different databases. Schema integration also requires a way to define how data are mapped from the local schema representation at each database, to the global schema. This step can be done by defining views at each site which, transform data from the local schema to the global schema. Data in the global schema is then treated as the union of the global views at the individual site. This approach is called the **global-as-view (GAV)** approach.

Consider an example with two sites which store student information in two different ways:

- Site s_1 which uses the relation $student1(ID, name, dept_name)$, and the relation $studentCreds(ID, tot_cred)$.
- Site s_2 which uses the relation $student2(ID, name, tot_cred)$, and the relation $studentDept(ID, dept_name)$.

Let the global schema chosen be $student(ID, name, dept_name, tot_cred)$.

Then, the global schema view at site *s1* would be defined as the view:

```
create view student_s1(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student1, studentCreds
where student1.ID= studentCreds.ID;
```

While the global schema view at site *s2* would be defined as the view:

```
create view student_s2(ID, name, dept_name, tot_cred) as
select ID, name, dept_name, tot_cred
from student2, studentDept
where student2.ID= studentDept.ID;
```

Finally, the global schema *student* would be defined as the union of *student_s1* and *student_s2*.

Note that with the above view definition, a query on the global schema relation *student* can be easily translated into queries on the local schema relations at the sites *s1* and *s2*. It is harder to translate updates on the global schema into updates on the local schema, since there may not be a unique way to do so as discussed in Section 4.2.

There are more complex mapping schemes that are designed to deal with duplication of information across sites and to allow translation of updates on the global schema into updates on the local schema. The **local-as-view (LAV)** approach, which defines local data in each site as a view on a conceptual unified global relation, is one such approach.

Consider for example a situation where the *student* relation is partitioned between two sites based on the *dept_name* attribute, with all students in the “Comp. Sci.” department at site *s3* and all students in other departments in site *s4*. This can be specified using the local-as-view approach by defining the relations *student_s3* and *student_s4*, which are actually stored at the sites *s3* and *s4*, as equivalent to views defined on the global relation *student*.

```
create view student_s3 as
select *
from student
where student.dept_name = 'Comp. Sci.';
```

and

```
create view student_s4 as
select *
from student
where student.dept_name != 'Comp. Sci.';
```

With this extra information, the query optimizer can figure out that a query that seeks to retrieve students in the Comp. Sci. department need only be executed at site s_3 and need not be executed at site s_4 . More information on schema integration may be found in the bibliographic notes for this chapter, available online.

The second task in providing a unified view of data from multiple sources lies in dealing with differences in data types and values. For example, the data types used in one system may not be supported by other systems, and translation between types may not be simple. Even for identical data types, problems may arise from the physical representation of data: One system may use 8-bit ASCII, while another may use 16-bit Unicode; floating-point representations may differ; integers may be represented in *big-endian* or *little-endian* form. At the semantic level, an integer value for length may be inches in one system and millimeters in another; when integrating the data, a single representation must be used, and values converted to the chosen representation. Mapping between types can be done as part of the view definitions that translate data between the local schemas and the global schema.

A deeper problem is that the same conceptual entity may have different names in different systems. For example, a system based in the United States may refer to the city “Cologne,” whereas one in Germany refers to it as “Köln.” One approach to deal with this problem is to have a globally unique naming system, and map values to the unique names as part of the view definitions used for schema mappings. For example, the International Standards Organization has a unique code for country names, and for states/provinces within the countries. The GeoNames database (www.geonames.org) provides unique names for several million locations such as cities, geographical features, roads, buildings, and so forth.

When such standard naming systems are not available, some systems allow the specification of name equivalences; for example, such a system could allow a user to say that “Cologne” is the same as “Köln”. This approach is used in the [Linked Data](#) project, which supports the integration of a very large number of databases that use the RDF representation of data (the RDF representation is described in Section 8.1.4). However, querying is more complicated in such a scenario.

In our description of the view definitions above, we assumed that data are stored in local databases, and the view definitions are used to provide a global view of the data, without actually materializing the data in the global schema. However, such views can also be used to materialize the data in the global schema, which can then be stored in a data warehouse. In the latter case, updates on underlying data must be propagated to the data warehouse.

22.9.3 Query Processing Across Multiple Data Sources

A naive way to execute a query that accesses data from multiple data sources is to fetch all required data to one database, which then executes the query. But suppose, for example, that the query has a selection condition that is satisfied by only one or a few records out of a large relation. If the data source allows the selection to be performed at

the data source, it makes no sense to retrieve the entire relation; instead, the selection operation should be performed at the data source, while other operations, if any, may be performed at the database where the query was issued.

In general, different data sources may support different query capabilities. For example, if the source is a data storage system, it may support selections on key attributes only. Web data sources may restrict which fields selections are allowed on and may additionally require that selections be present on certain fields. On the other hand, if the source is a database that supports SQL, operations such as join or aggregation could be performed at the source and only the result brought over to the database that issues the query. In general, queries may have to be broken up and performed partly at the data source and partly at the site issuing the query.

The cost of processing a query that accesses multiple data sources depends on the local execution costs, as well as on the data transfer cost. If the network is a low bandwidth wide-area network, particular attention must be paid to minimizing data transfer.

In this section, we study issues in distributed query processing and optimization.

22.9.3.1 Join Locations and Join Ordering

Consider the following relational-algebra expression:

$$r_1 \bowtie r_2 \bowtie r_3$$

Assume that r_1 is stored at site S_1 , r_2 at S_2 , and r_3 at S_3 . Let S_I denote the site at which the query was issued. The system needs to produce the result at site S_I . Among the possible strategies for processing this query are these:

- Ship copies of all three relations to site S_I . Using the techniques of Chapter 16, choose a strategy for processing the entire query locally at site S_I .
- Ship a copy of the r_1 relation to site S_2 , and compute $temp_1 = r_1 \bowtie r_2$ at S_2 . Ship $temp_1$ from S_2 to S_3 , and compute $temp_2 = temp_1 \bowtie r_3$ at S_3 . Ship the result $temp_2$ to S_I .
- Devise strategies similar to the previous one, with the roles of S_1, S_2, S_3 exchanged.

There are several other possible strategies.

No one strategy is always the best one. Among the factors that must be considered are the volume of data being shipped, the cost of transmitting a block of data between a pair of sites, and the relative speed of processing at each site. Consider the first strategy. Suppose indices present at S_2 and S_3 are useful for computing the join. If we ship all three relations to S_I , we would need to either re-create these indices at S_I or use a different, possibly more expensive, join strategy. Re-creation of indices entails extra processing overhead and extra disk accesses. There are many variants of the second strategy, which process joins in different orders.

The cost of each of the strategies depends on the sizes of the intermediate results, the network transmission costs, and the costs of processing at each node. The query optimizer needs to choose the best strategy, based on cost estimates.

22.9.3.2 Semijoin Strategy

Suppose that we wish to evaluate the expression $r_1 \bowtie r_2$, where r_1 and r_2 are stored at sites S_1 and S_2 , respectively. Let the schemas of r_1 and r_2 be R_1 and R_2 . Suppose that we wish to obtain the result at S_1 . If there are many tuples of r_2 that do not join with any tuple of r_1 , then shipping r_2 to S_1 entails shipping tuples that fail to contribute to the result. We want to remove such tuples before shipping data to S_1 , particularly if network costs are high.

A possible strategy to accomplish all this is:

1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
2. Ship $temp_1$ from S_1 to S_2 .
3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2 .
4. Ship $temp_2$ from S_2 to S_1 .
5. Compute $r_1 \bowtie temp_2$ at S_1 . The resulting relation is the same as $r_1 \bowtie r_2$.

Before considering the efficiency of this strategy, let us verify that the strategy computes the correct answer. In step 3, $temp_2$ has the result of $r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$. In step 5, we compute:

$$r_1 \bowtie r_2 \bowtie \Pi_{R_1 \cap R_2}(r_1)$$

Since join is associative and commutative, we can rewrite this expression as:

$$(r_1 \bowtie \Pi_{R_1 \cap R_2}(r_1)) \bowtie r_2$$

Since $r_1 \bowtie \Pi_{(R_1 \cap R_2)}(r_1) = r_1$, the expression is, indeed, equal to $r_1 \bowtie r_2$, the expression we are trying to evaluate.

This strategy is called a **semijoin strategy**, after the semijoin operator of the relational algebra, denoted $\ltimes$, which we saw in Section 16.4.4. The *natural semijoin* of r_1 with r_2 , denoted $r_1 \ltimes r_2$, is defined as:

$$r_1 \ltimes r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie r_2)$$

Thus, $r_1 \ltimes r_2$ selects those tuples of relation r_1 that contributed to $r_1 \bowtie r_2$. In step 3, $temp_2 = r_2 \ltimes r_1$. The semijoin operation is easily extended to theta-joins. The *theta semijoin* of r_1 with r_2 , denoted $r_1 \ltimes_\theta r_2$, is defined as:

$$r_1 \ltimes_\theta r_2 \stackrel{\text{def}}{=} \Pi_{R_1}(r_1 \bowtie_\theta r_2)$$

For joins of several relations, the semijoin strategy can be extended to a series of semijoin steps. It is the job of the query optimizer to choose the best strategy based on cost estimates.

This strategy is particularly advantageous when relatively few tuples of r_2 contribute to the join. This situation is likely to occur if r_1 is the result of a relational-algebra expression involving selection. In such a case, $temp_2$, that is, $r_2 \bowtie r_1$, may have significantly fewer tuples than r_2 . The cost savings of the strategy result from having to ship only $r_2 \bowtie r_1$, rather than all of r_2 , to S_1 .

Some additional cost is incurred in shipping $temp_1$, that is $\Pi_{R_1 \cap R_2}(r_1)$ to S_2 . If a sufficiently small fraction of tuples in r_2 contribute to the join, the overhead of shipping $temp_1$ will be dominated by the savings of shipping only a fraction of the tuples in r_2 . The overhead of sending $temp_1$ tuples from s_1 to s_2 can be reduced as follows. For the purpose of optimization of join processing, the semijoin operation can be implemented in a manner that overapproximates the true semijoin result. That is, the result should contain all the tuples in the actual semijoin result, but it may contain a few extra tuples. The extra tuples will get eliminated later by the join operation.

An efficient overapproximation of the semijoin result can be computed by using a probabilistic data structure called a **Bloom filter**, which uses bitmaps. Bloom filters are described in more detail in Section 24.1. To implement $r_2 \bowtie r_1$, a Bloom filter with a bitmap b of size m , initialized with all bits set to 0 is used. The join attributes of each tuple of r_1 are hashed to a value in the range $0 \dots (m - 1)$, and the corresponding bit of b is set to 1. The bitmap b , which is much smaller than the relation r_1 , can now be sent to the site containing r_2 . There, the same hash function is computed on the join attributes of each tuple of r_2 . If the corresponding bit is set to 1 in b , that r_2 tuple is accepted (added to the *result* relation), and otherwise it is rejected.

Note that it is possible for different join attribute values, say v_1 and v_2 to have the same hash value; even if r_1 has a tuple with value v_1 , but does not have any tuple with value v_2 , the result of the above procedure may include r_2 tuples whose join attribute value is v_2 . Such a situation is referred to as a **false positive**. However, if v_1 is present in r_1 , the technique will never reject a tuple in r_2 that has join attribute value v_1 , which is important for correctness.

The *result* relation computed above, which is a superset of or equal to $r_2 \bowtie r_1$, is sent to site s_1 . The join $r_1 \bowtie result$ is then computed at site s_1 to get the required join result. False positives may result in extra tuples in *result* that are not present in $r_2 \bowtie r_1$, but such tuples would be eliminated by the join.

To keep the probability of false positives low, the number of bits in the Bloom filter is usually set to a few times the estimated number of distinct join attribute values. Further, it is possible to use k independent hash functions, for some $k > 1$, to identify k bit positions for a given value, and set all of them to 1 when creating the bitmap. When querying it with a given value, the same k hash functions are used to identify k bit locations, and the value is determined to be absent if even one of the k bits has a 0 value. For example, if the bitmap has $10n$ bits, where n is the number of distinct join

attribute values, and $k = 7$ hash functions are used, the false positive rate would be about 1%.

22.9.3.3 Distributed Query Optimization

Several extensions need to be made to existing query optimization techniques in order to optimize distributed query plans.

The first extension is to record the location of data as a *physical property* of the data; recall that optimizers already deal with other physical properties such as the sort order of results. Just as the sort operation is used to create different sort orders, an exchange operation is used to transfer data between different sites.

The second extension is to track where an operator is executed; optimizers already consider different algorithms, such as hash join or merge join, for a given logical operator, in this case, the join operator. The optimizer is extended to additionally consider alternative sites for execution of each algorithm. Note that to execute an operator at a given site, its inputs must satisfy the physical property of being located at that site.

The third extension is to consider semijoin operations to reduce data transfer costs. Semijoin operations can be introduced as logical transformation rules; however, if done naively, the search space increases greatly, making this approach infeasible. Optimization cost can be reduced by restricting, as a heuristic, semijoins to be applied only on database tables, and never on intermediate join results.

A fourth extension is to use schema information to restrict the set of nodes at which a query needs to be executed. Recall from Section 22.9.2 that the local-as-view approach can be used to specify that a relation is partitioned in a particular way. In the example we saw there, site s_3 contains all student tuples with *dept_name* being Comp. Sci., while s_4 contains all the other student tuples. Suppose a query has a selection “*dept_name*='Comp. Sci.'” on *student*; then, the optimizer should recognize that there is no need to involve site s_4 when executing this query.

As another example, if the student data at site s_5 is a replica of the data at site s_3 , then the optimizer can choose to execute the query at either of the sites, depending on which is cheaper; there is no need to execute the query at both sites.

22.9.4 Distributed Directory Systems

A directory is a listing of information about some class of objects such as persons. Directories can be used to find information about a specific object, or in the reverse direction to find objects that meet a certain requirement. Several **directory access protocols** have been developed to provide a standardized way of accessing data in a directory.

A very widely used distributed directory system is the internet **Domain Name Service (DNS)** system, which provides a standardized way to map domain names (such as *db-book.com* or *www.cs.yale.edu*, to the IP addresses of the machines. (Although users see only the domain names, the underlying network routes messages based on IP addresses, and hence a way to convert domain names to IP addresses is critical for

the functioning of the internet.) The **Lightweight Directory Access Protocol (LDAP)** is another very widely used protocol designed for storing organizational data.

Data stored in directories can be represented in the relational model, stored in a relational database, and accessed through standard protocols such as JDBC or ODBC. The question then is, why come up with a specialized protocol for accessing directory information? There are several reasons.

- First, directory access protocols are simplified protocols that cater to a limited type of access to data. They evolved in parallel with the database access protocol standards.
- Second, directory systems were designed to support a hierarchical naming system for objects, similar to file system directory names. Such a naming system is important in many applications. For example, all computers whose names end in `yale.edu` belong to Yale, while those whose names end in `iitb.ac.in` belong to IIT Bombay. Within the `yale.edu` domain, there are subdomains such as `cs.yale.edu`, which corresponds to the CS department in Yale, and `math.yale.edu` which corresponds to the Math department at Yale.
- Third, and most important from a distributed systems perspective, the data in a distributed directory system are stored and controlled in a distributed, hierarchical, manner.

For example, a DNS server at Yale would store information about names of computers at Yale, along with associated information such as their IP addresses. Similarly, DNS servers at Lehigh and IIT Bombay would store information about computers in their respective domains. The DNS servers store information in a hierarchical fashion; for example, the information provided by the Yale DNS server may be stored in a distributed fashion across subdomains at Yale, such as the CS and Math DNS servers.

Distributed directory systems automatically forward queries submitted at a site to the site where the required information is actually stored, to give a unified view of data to users and applications.

Further, distributed directory implementations typically support replication to ensure the availability of data even if some nodes have failed.

Another example of usage of directory systems is for organization data. Such systems store information about employees, such as the employee identifier, name, email, organization unit (such as department), room number, phone number, and (encrypted) password of each employee. The schema of such organizational data are standardized as part of the Lightweight Directory Access Protocol (LDAP). Directory systems based on the LDAP protocol are widely used to authenticate users, using the encrypted passwords stored for each user. (More information about the LDAP data representation may be found in Section 25.5.)

heterogeneous distributed databases. Replication with weak degrees of consistency is discussed in Section 23.6.

Most techniques for dealing with distributed data require the use of coordinators to ensure consistent and efficient transaction processing. In Section 23.7 we discuss how coordinators can be chosen in a distributed fashion, robust to failures. Finally, Section 23.8 describes the distributed consensus problem, outlines solutions for the problem, and then discusses how these solutions can be used to implement fault-tolerant services by means of replication of a log.

23.1 Distributed Transactions

Access to the various data items in a distributed system is usually accomplished through transactions, which must preserve the ACID properties (Section 17.1). There are two types of transaction that we need to consider. The **local transactions** are those that access and update data in only one local database; the **global transactions** are those that access and update data in several local databases. Ensuring the ACID properties of the local transactions can be done as described in Chapter 17, Chapter 18, and Chapter 19. However, for global transactions, this task is much more complicated, since several nodes may be participating in the execution of the transaction. The failure of one of these nodes, or the failure of a communication link connecting these nodes, may result in erroneous computations.

In this section, we study the system structure of a distributed database and its possible failure modes. In later sections, we discuss how to ensure ACID properties are satisfied in a distributed database, despite failures. We reemphasize that these failure modes occur with parallel databases as well, and the techniques we describe are equally applicable to parallel databases.

23.1.1 System Structure

We now consider a system structure with multiple nodes, each of which can fail independently of the others. We note that the nodes may be within a single data center, corresponding to a parallel database system, or geographically distributed, in a distributed database system. The system structure is similar in either case; the problems with respect to transaction isolation and atomicity are the same in both cases, as are the solutions.

We note that the system structure we consider here is not applicable to a shared-memory parallel database system whose components do not have independent modes of failures. In such systems either the whole system is up, or the whole system is down. Further, there is usually only one transaction log used for recovery. Concurrency control and recovery techniques that are designed for centralized database systems can be used in such systems, and are preferable to techniques described in this chapter.

Each node has its own *local* transaction manager, whose function is to ensure the ACID properties of those transactions that execute at that node. The various trans-

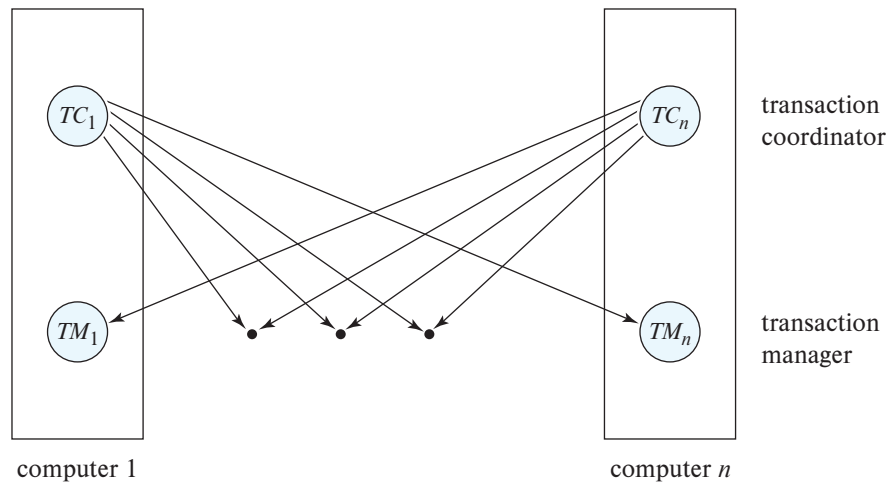


Figure 23.1 System architecture.

action managers cooperate to execute global transactions. To understand how such a manager can be implemented, consider an abstract model of a transaction system, in which each node contains two subsystems:

- The **transaction manager** manages the execution of those transactions (or subtransactions) that access data stored in the node. Note that each such transaction may be either a local transaction (i.e., a transaction that executes at only that node) or part of a global transaction (i.e., a transaction that executes at several nodes).
- The **transaction coordinator** coordinates the execution of the various transactions (both local and global) initiated at that node.

The overall system architecture appears in Figure 23.1.

The structure of a transaction manager is similar in many respects to the structure of a centralized system. Each transaction manager is responsible for:

- Maintaining a log for recovery purposes.
- Participating in an appropriate concurrency-control scheme to coordinate the concurrent execution of the transactions executing at that node.

As we shall see, we need to modify both the recovery and concurrency schemes to accommodate the distributed execution of transactions.

The transaction coordinator subsystem is not needed in the centralized environment, since a transaction accesses data at only a single node. A transaction coordinator, as its name implies, is responsible for coordinating the execution of all the transactions initiated at that node. For each such transaction, the coordinator is responsible for:

- Starting the execution of the transaction.
- Breaking the transaction into a number of subtransactions and distributing these subtransactions to the appropriate nodes for execution.
- Coordinating the termination of the transaction, which may result in the transaction being committed at all nodes or aborted at all nodes.

23.1.2 System Failure Modes

A distributed system may suffer from the same types of failure that a centralized system does (e.g., software errors, hardware errors, or disk crashes). There are, however, additional types of failure with which we need to deal in a distributed environment. The basic failure types are:

- Failure of a node.
- Loss of messages.
- Failure of a communication link.
- Network partition.

The loss or corruption of messages is always a possibility in a distributed system. The system uses transmission-control protocols, such as TCP/IP, to handle such errors. Information about such protocols may be found in standard textbooks on networking.

However, if two nodes *A* and *B* are not directly connected, messages from one to the other must be *routed* through a sequence of communication links. If a communication link fails, messages that would have been transmitted across the link must be rerouted. In some cases, it is possible to find another route through the network, so that the messages are able to reach their destination. In other nodes, a failure may result in there being no connection between some pairs of nodes. A system is **partitioned** if it has been split into two (or more) subsystems, called **partitions**, that lack any connection between them. Note that, under this definition, a partition may consist of a single node.

23.2 Commit Protocols

If we are to ensure atomicity, all the nodes in which a transaction *T* executed must agree on the final outcome of the execution. *T* must either commit at all nodes, or it must abort at all nodes. To ensure this property, the transaction coordinator of *T* must execute a commit protocol.

Among the simplest and most widely used commit protocols is the **two-phase commit protocol (2PC)**, which is described in Section 23.2.1.

23.2.1 Two-Phase Commit

We first describe how the two-phase commit protocol (2PC) operates during normal operation, then describe how it handles failures and finally how it carries out recovery and concurrency control.

Consider a transaction T initiated at node N_i , where the transaction coordinator is C_i .

23.2.1.1 The Commit Protocol

When T completes its execution—that is, when all the nodes at which T has executed inform C_i that T has completed— C_i starts the 2PC protocol.

- Phase 1.** C_i adds the record $\langle \text{prepare } T \rangle$ to the log and forces the log onto stable storage. It then sends a **prepare T** message to all nodes at which T executed. On receiving such a message, the transaction manager at that node determines whether it is willing to commit its portion of T . If the answer is no, it adds a record $\langle \text{no } T \rangle$ to the log and then responds by sending an **abort T** message to C_i . If the answer is yes, it adds a record $\langle \text{ready } T \rangle$ to the log and forces the log (with all the log records corresponding to T) onto stable storage. The transaction manager then replies with a **ready T** message to C_i .
- Phase 2.** When C_i receives **ready** responses to the **prepare T** message from all the nodes, or when it receives an **abort T** message from at least one participant node, C_i can determine whether the transaction T can be committed or aborted. Transaction T can be committed if C_i received a **ready T** message from all the participating nodes. Otherwise, transaction T must be aborted. Depending on the verdict, either a record $\langle \text{commit } T \rangle$ or a record $\langle \text{abort } T \rangle$ is added to the log and the log is forced onto stable storage. At this point, the fate of the transaction has been sealed.

Following this point, the coordinator sends either a **commit T** or an **abort T** message to all participating nodes. When a node receives that message, it records the result (either $\langle \text{commit } T \rangle$ or $\langle \text{abort } T \rangle$) in its log, and correspondingly either commits or aborts the transaction.

Since nodes may fail, the coordinator cannot wait indefinitely for responses from all the nodes. Instead, when a prespecified interval of time has elapsed since the **prepare T** message was sent out, if any node has not responded to the coordinator, the coordinator can decide to abort the transaction; the steps described for aborting the transaction must be followed, just as if a node had sent an **abort** message for the transaction.

Figure 23.2 shows an instance of successful execution of 2PC for a transaction T , with two nodes, N_1 and N_2 , that are both willing to commit transaction T . If any of the

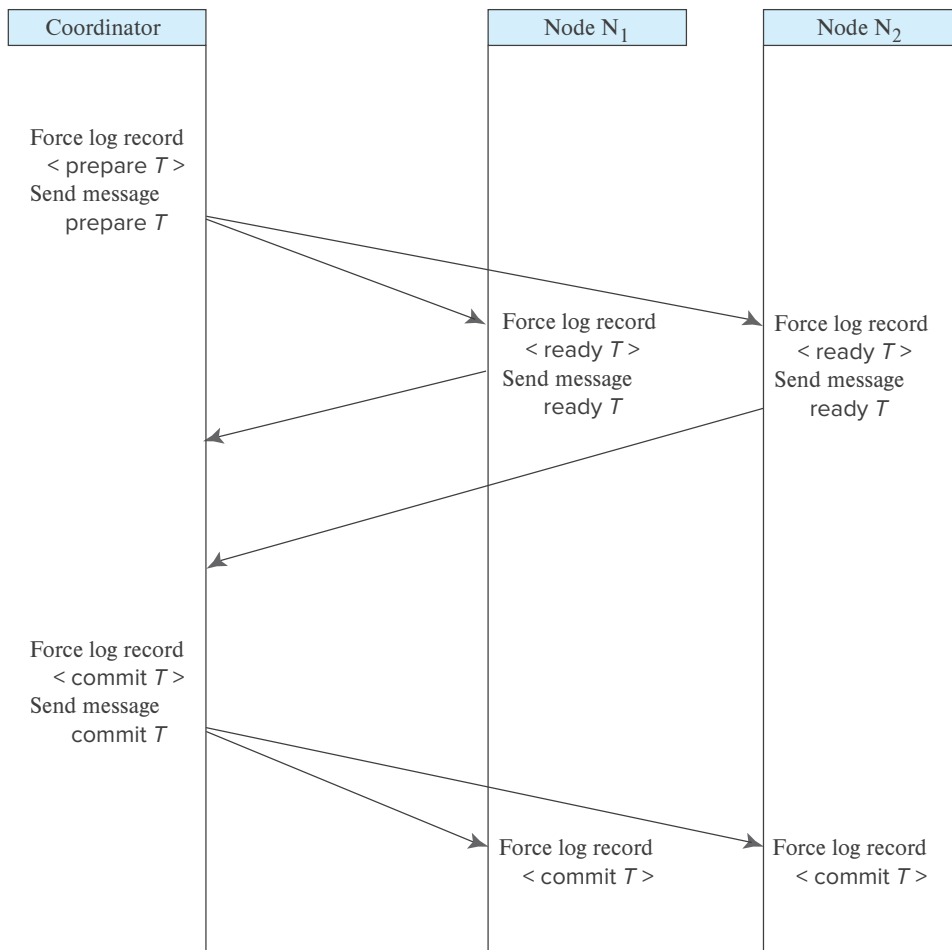


Figure 23.2 Successful execution of 2PC.

nodes sends a `no T` message, the coordinator will send an `abort T` message to all the nodes, which will then abort the transaction.

A node at which T executed can unconditionally abort T at any time before it sends the message `ready T` to the coordinator. Once the `<ready T>` log record is written, the transaction T is said to be in the **ready state** at the node. The `ready T` message is, in effect, a promise by a node to follow the coordinator's order to commit T or to abort T . To make such a promise, the needed information must first be stored in stable storage. Otherwise, if the node crashes after sending `ready T`, it may be unable to make good on its promise. Further, locks acquired by the transaction must continue to be held until the transaction completes, even if there is an intervening node failure, as we shall see in Section 23.2.1.3.

Since unanimity is required to commit a transaction, the fate of T is sealed as soon as at least one node responds **abort** T . Since the coordinator node N_i is one of the nodes at which T executed, the coordinator can decide unilaterally to **abort** T . The final verdict regarding T is determined at the time that the coordinator writes that verdict (commit or abort) to the log and forces that verdict to stable storage.

In some implementations of the 2PC protocol, a node sends an **acknowledge** T message to the coordinator at the end of the second phase of the protocol. When the coordinator receives the **acknowledge** T message from all the nodes, it adds the record **<complete** T to the log. Until this step, the coordinator cannot forget about the commit or abort decision on T , since a node may ask for the decision. (A node that has not received a commit or abort for transaction T , perhaps due to a network failure or temporary node failure, may send such a request to the coordinator.) After this step, the coordinator can discard information about transaction T .

23.2.1.2 Handling of Failures

The 2PC protocol responds in different ways to various types of failure:

- **Failure of a participating node.** If the coordinator C_i detects that a node has failed, it takes these actions: If the node fails before responding with a **ready** T message to C_i , the coordinator assumes that it responded with an **abort** T message. If the node fails after the coordinator has received the **ready** T message from the node, the coordinator executes the rest of the commit protocol in the normal fashion, ignoring the failure of the node.

When a participating node N_k recovers from a failure, it must examine its log to determine the fate of those transactions that were in the midst of execution when the failure occurred. Let T be one such transaction. We consider each of the possible cases:

- The log contains a **<commit** T record. In this case, the node executes **redo**(T).
- The log contains an **<abort** T record. In this case, the node executes **undo**(T).
- The log contains a **<ready** T record. In this case, the node must consult C_i to determine the fate of T . If C_i is up, it notifies N_k regarding whether T committed or aborted. In the former case, it executes **redo**(T); in the latter case, it executes **undo**(T). If C_i is down, N_k must try to find the fate of T from other nodes. It does so by sending a **querystatus** T message to all the nodes in the system. On receiving such a message, a node must consult its log to determine whether T has executed there, and if T has, whether T committed or aborted. It then notifies N_k about this outcome. If no node has the appropriate information (i.e., whether T committed or aborted), then N_k can neither abort nor commit T . The decision concerning T is postponed until N_k can obtain the needed

information. Thus, N_k must periodically resend the `querystatus` message to the other nodes. It continues to do so until a node that contains the needed information has recovered. Note that the node at which C_i resides always has the needed information.

- The log contains no control records (`abort`, `commit`, `ready`) concerning T . Thus, we know that N_k failed before responding to the `prepare T` message from C_i . Since the failure of N_k precludes the sending of such a response, by our algorithm C_i must abort T . Hence, N_k must execute `undo(T)`.
- **Failure of the coordinator.** If the coordinator fails in the midst of the execution of the commit protocol for transaction T , then the participating nodes must decide the fate of T . We shall see that, in certain cases, the participating nodes cannot decide whether to commit or abort T , and therefore these nodes must wait for the recovery of the failed coordinator.
 - If an active node contains a `<commit T>` record in its log, then T must be committed.
 - If an active node contains an `<abort T>` record in its log, then T must be aborted.
 - If some active node does *not* contain a `<ready T>` record in its log, then the failed coordinator C_i cannot have decided to commit T , because a node that does not have a `<ready T>` record in its log cannot have sent a `ready T` message to C_i . However, the coordinator may have decided to abort T , but not to commit T . Rather than wait for C_i to recover, it is preferable to abort T .
 - If none of the preceding cases holds, then all active nodes must have a `<ready T>` record in their logs, but no additional control records (such as `<abort T>` or `<commit T>`). Since the coordinator has failed, it is impossible to determine whether a decision has been made, and if one has, what that decision is, until the coordinator recovers. Thus, the active nodes must wait for C_i to recover.

Since the fate of T remains in doubt, T may continue to hold system resources. For example, if locking is used, T may hold locks on data at active nodes. Such a situation is undesirable, because it may be hours or days before C_i is again active. During this time, other transactions may be forced to wait for T . As a result, data items may be unavailable not only on the failed node (C_i), but on active nodes as well. This situation is called the **blocking problem**, because T is blocked pending the recovery of node C_i .
- **Network partition.** When a network partition occurs, two possibilities exist:
 1. The coordinator and all its participants remain in one partition. In this case, the failure has no effect on the commit protocol.

2. The coordinator and its participants belong to several partitions. From the viewpoint of the nodes in one of the partitions, it appears that the nodes in other partitions have failed. Nodes that are not in the partition containing the coordinator simply execute the protocol to deal with the failure of the coordinator. The coordinator and the nodes that are in the same partition as the coordinator follow the usual commit protocol, assuming that the nodes in the other partitions have failed.

Thus, the major disadvantage of the 2PC protocol is that coordinator failure may result in blocking, where a decision either to commit or to abort T may have to be postponed until C_i recovers. We discuss how to remove this limitation shortly, in Section 23.2.2.

23.2.1.3 Recovery and Concurrency Control

When a failed node restarts, we can perform recovery by using, for example, the recovery algorithm described in Section 19.4. To deal with distributed commit protocols, the recovery procedure must treat **in-doubt transactions** specially; in-doubt transactions are transactions for which a **<ready T >** log record is found, but neither a **<commit T >** log record nor an **<abort T >** log record is found. The recovering node must determine the commit–abort status of such transactions by contacting other nodes, as described in Section 23.2.1.2.

If recovery is done as just described, however, normal transaction processing at the node cannot begin until all in-doubt transactions have been committed or rolled back. Finding the status of in-doubt transactions can be slow, since multiple nodes may have to be contacted. Further, if the coordinator has failed, and no other node has information about the commit–abort status of an incomplete transaction, recovery potentially could become blocked if 2PC is used. As a result, the node performing restart recovery may remain unusable for a long period.

To circumvent this problem, recovery algorithms typically provide support for noting lock information in the log. (We are assuming here that locking is used for concurrency control.) Instead of writing a **<ready T >** log record, the algorithm writes a **<ready T, L >** log record, where L is a list of all write locks held by the transaction T when the log record is written. At recovery time, after performing local recovery actions, for every in-doubt transaction T , all the write locks noted in the **<ready T, L >** log record (read from the log) are reacquired.

After lock reacquisition is complete for all in-doubt transactions, transaction processing can start at the node, even before the commit–abort status of the in-doubt transactions is determined. The commit or rollback of in-doubt transactions proceeds concurrently with the execution of new transactions. Thus, node recovery is faster and never gets blocked. Note that new transactions that have a lock conflict with any write locks held by in-doubt transactions will be unable to make progress until the conflicting in-doubt transactions have been committed or rolled back.

23.2.2 Avoiding Blocking During Commit

The blocking problem of 2PC is a serious concern for system designers, since the failure of a coordinator node could lead to blocking of a transaction that has acquired locks on a frequently used data item, which in turn prevents other transactions that need to acquire a conflicting lock from completing their execution.

By involving multiple nodes in the commit decision step of 2PC, it is possible to avoid blocking as long as a majority of the nodes involved in the commit decision are alive and can communicate with each other. This is done by using the idea of fault-tolerant distributed consensus. Details of distributed consensus are discussed in detail later, in Section 23.8, but we outline the problem and sketch a solution approach below.

The **distributed consensus problem** is as follows: A set of n nodes need to agree on a decision; in this case, whether or not to commit a particular transaction. The inputs to make the decision are provided to all the nodes, and then each node votes on the decision; in the case of 2PC, the decision is on whether or not to commit a transaction. The key goal of protocols for achieving distributed consensus is that the decision should be made in such a way that all nodes will “learn” the same value for the decision (i.e., all nodes will learn that the transaction is to be committed, or all nodes will learn that the transaction is to be aborted), even if some nodes fail during the execution of the protocol, or there are network partitions. Further, the distributed consensus protocol should not block, as long as a majority of the nodes participating remain alive and can communicate with each other.

There are several protocols for distributed consensus, two of which are widely used today (Paxos and Raft). We study distributed consensus in Section 23.8. A key idea behind these protocols is the idea of a vote, which succeeds only if a majority of the participating nodes agree on a particular decision.

Given an implementation of distributed consensus, the blocking problem due to coordinator failure can be avoided as follows: Instead of the coordinator locally deciding to commit or abort a transaction, it initiates the distributed consensus protocol, requesting that the value “committed” or “aborted” be assigned to the transaction T . The request is sent to all the nodes participating in the distributed consensus, and the consensus protocol is then executed by those nodes. Since the protocol is fault tolerant, it will succeed even if some nodes fail, as long as a majority of the nodes are up and remain connected. The transaction can be declared committed by the coordinator only after the consensus protocol completes successfully.

There are two possible failure scenarios:

- *The coordinator fails at any stage before informing all participating nodes of the commit or abort status of a transaction T .*

In this case, a new coordinator is chosen (we will see how to do so in Section 23.7). The new coordinator checks with the nodes participating in the distributed consensus to see if a decision was made, and if so informs the 2PC participants

of the decision. A majority of the nodes participating in consensus must respond, to check if a decision was made or not; the protocol will not block as long as the failed/disconnected nodes are in a minority.

If no decision was made earlier for transaction T , the new coordinator again checks with the 2PC participants to check if they are ready to commit or wish to abort the transaction, and follows the usual coordinator protocol based on their responses. As before, if no response is received from a participant, the new coordinator may choose to abort T .

- *The distributed consensus protocol fails to reach a decision.*

Failure of the protocol can occur due to the failure of some participating nodes. It could also occur because of conflicting requests, none of which gets a majority of “votes” during the consensus protocol. For 2PC, the request normally comes from a single coordinator, so such a conflict is unlikely. However, conflicting requests can arise in rare cases if a coordinator fails after sending out a commit message, but its commit message is delivered late; meanwhile, a new 2PC coordinator makes an abort decision since it could not reach some participating nodes. Even with such a conflict, the distributed consensus protocol guarantees that only one of the commit or abort requests can succeed, even in the presence of failures. But if some nodes are down, and neither the commit nor the abort request gets a majority vote from nodes participating in the distributed consensus, it is possible for the protocol to fail to reach a decision.

Regardless of the reason, if the distributed consensus protocol fails to reach a decision, the new coordinator just re-initiates the protocol.

Note that in the event of a network partition, a node that gets disconnected from the majority of the nodes participating in consensus may not learn about a decision, even if a decision was successfully made. Thus, transactions running at such a node may be blocked.

Failure of 2PC participants could make data unavailable, in the absence of replication. Distributed consensus can also be used to keep replicas of a data item in a consistent state, as we explain later in Section 23.8.4.

The idea of distributed consensus to make 2PC nonblocking was proposed in the 1980s; it is used, for example, in the Google Spanner distributed database system.

The **three-phase commit (3PC)** protocol is an extension of the two-phase commit protocol that avoids the blocking problem under certain assumptions. One variant of the protocol avoids blocking as long as network partitions do not occur, but it may lead to inconsistent decisions in the event of a network partition. Extensions of the protocol that work safely under network partitioning were developed subsequently. The idea behind these extensions is similar to the majority voting idea of distributed consensus, but the protocols are specifically tailored for the task of atomic commit.

23.2.3 Alternative Models of Transaction Processing

With two-phase commit, participating nodes agree to let the coordinator decide the fate of a transaction, and are forced to wait for the decision of the coordinator, while holding locks on updated data items. While such loss of autonomy may be acceptable within an organization, no organization would be willing to force its computers to wait, potentially for a long time, while a computer at another organization makes the decision.

In this section, we describe how to use *persistent messaging* to avoid the problem of distributed commit. To understand persistent messaging, consider how one might transfer funds between two different banks, each with its own computer. One approach is to have a transaction span the two nodes and use two-phase commit to ensure atomicity. However, the transaction may have to update the total bank balance, and blocking could have a serious impact on all other transactions at each bank, since almost all transactions at the bank would update the total bank balance.

In contrast, consider how funds transfer by a bank check occurs. The bank first deducts the amount of the check from the available balance and prints out a check. The check is then physically transferred to the other bank where it is deposited. After verifying the check, the bank increases the local balance by the amount of the check. The check constitutes a message sent between the two banks. So that funds are not lost or incorrectly increased, the check must not be lost and must not be duplicated and deposited more than once. When the bank computers are connected by a network, persistent messages provide the same service as the check (but much faster).

Persistent messages are messages that are guaranteed to be delivered to the recipient exactly once (neither less nor more), regardless of failures, if the transaction sending the message commits, and are guaranteed to not be delivered if the transaction aborts. Database recovery techniques are used to implement persistent messaging on top of the normal network channels, as we shall see shortly. In contrast, regular messages may be lost or may even be delivered multiple times in some situations.

Error handling is more complicated with persistent messaging than with two-phase commit. For instance, if the account where the check is to be deposited has been closed, the check must be sent back to the originating account and credited back there. Both nodes must, therefore, be provided with error-handling code, along with code to handle the persistent messages. In contrast, with two-phase commit, the error would be detected by the transaction, which would then never deduct the amount in the first place.

The types of exception conditions that may arise depend on the application, so it is not possible for the database system to handle exceptions automatically. The application programs that send and receive persistent messages must include code to handle exception conditions and bring the system back to a consistent state. For instance, it is not acceptable to just lose the money being transferred if the receiving account has been closed; the money must be credited back to the originating account, and if that is not possible for some reason, humans must be alerted to resolve the situation manually.

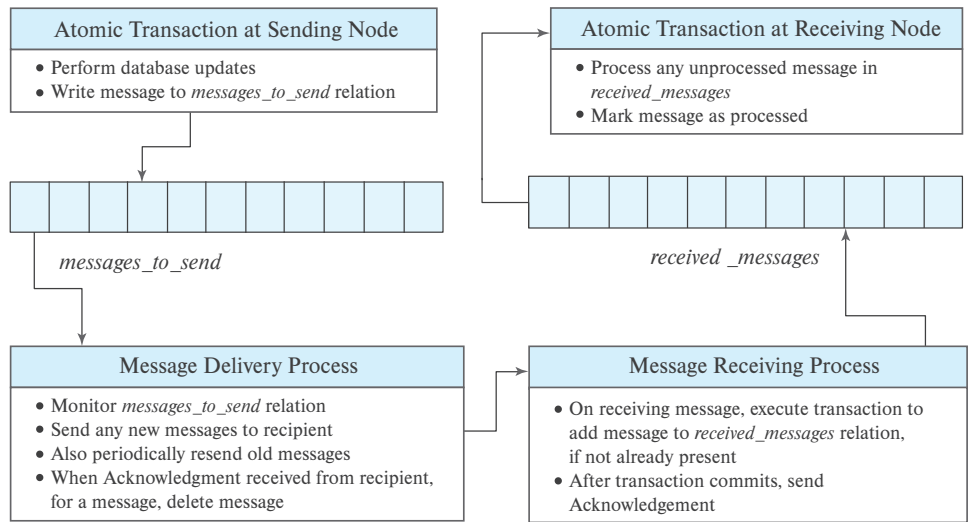


Figure 23.3 Implementation of persistent messaging.

There are many applications where the benefit of eliminating blocking is well worth the extra effort to implement systems that use persistent messages. In fact, few organizations would agree to support two-phase commit for transactions originating outside the organization, since failures could result in blocking of access to local data. Persistent messaging therefore plays an important role in carrying out transactions that cross organizational boundaries.

We now consider the **implementation** of persistent messaging. Persistent messaging can be implemented on top of an unreliable messaging infrastructure, which may lose messages or deliver them multiple times. Figure 23.3 shows a summary of the implementation, which is described in detail next.

- **Sending node protocol.** When a transaction wishes to send a persistent message, it writes a record containing the message in a special relation *messages_to_send*, instead of directly sending out the message. The message is also given a unique message identifier. Note that this relation acts as a message *outbox*.

A *message delivery process* monitors the relation, and when a new message is found, it sends the message to its destination. The usual database concurrency-control mechanisms ensure that the system process reads the message only after the transaction that wrote the message commits; if the transaction aborts, the usual recovery mechanism would delete the message from the relation.

The message delivery process deletes a message from the relation only after it receives an acknowledgment from the destination node. If it receives no acknowledgment from the destination node, after some time it sends the message again. It repeats this until an acknowledgment is received. In case of permanent failures,

the system will decide, after some period of time, that the message is undeliverable. Exception handling code provided by the application is then invoked to deal with the failure.

Writing the message to a relation and processing it only after the transaction has committed ensures that the message will be delivered if and only if the transaction commits. Repeatedly sending it guarantees it will be delivered even if there are (temporary) system or network failures.

- **Receiving node protocol.** When a node receives a persistent message, it runs a transaction that adds the message to a special *received_messages* relation, provided it is not already present in the relation (the unique message identifier allows duplicates to be detected). The relation has an attribute to indicate if the message has been processed, which is set to false when the message is inserted in the relation. Note that this relation acts as a message *inbox*.

After the transaction commits, or if the message was already present in the relation, the receiving node sends an acknowledgment back to the sending node.

Note that sending the acknowledgment before the transaction commits is not safe, since a system failure may then result in loss of the message. Checking whether the message has been received earlier is essential to avoid multiple deliveries of the message.

- **Processing of message.** Received messages must be processed to carry out the actions specified in the message. A process at the receiving node monitors the *received_messages* relation to check for messages that have not been processed. When it finds such a message, the message is processed, and as part of the same transaction that processes the message, the processed flag is set to true. This ensures that a message is processed exactly once after it is received.
- **Deletion of old messages.** In many messaging systems, it is possible for messages to get delayed arbitrarily, although such delays are very unlikely. Therefore, to be safe, the message must never be deleted from the *received_messages* relation. Deleting it could result in a duplicate delivery not being detected. But as a result, the *received_messages* relation may grow indefinitely. To deal with this problem, each message is given a timestamp, and if the timestamp of a received message is older than some cutoff, the message is discarded. All messages recorded in the *received_messages* relation that are older than the cutoff can be deleted.

Workflows provide a general model of distributed transaction processing involving multiple nodes and possibly human processing of certain steps, and they are supported by application software used by enterprises. For instance, as we saw in Section 9.6.1, when a bank receives a loan application, there are many steps it must take, including contacting external credit-checking agencies, before approving or rejecting a loan application. The steps, together, form a workflow. Persistent messaging forms the underlying basis for supporting workflows in a distributed environment.

node N_1 , T_2 follows T_1 , whereas, at N_2 , T_1 follows T_2 , resulting in a nonserializable global schedule. Indeed, even if there is no concurrency among global transactions (i.e., a global transaction is submitted only after the previous one commits or aborts), local serializability is not sufficient to ensure global serializability (see Practice Exercise 23.11).

Depending on the implementation of the local database systems, a global transaction may not be able to control the precise locking behavior of its local subtransactions. Thus, even if all local database systems follow two-phase locking, it may be possible only to ensure that each local transaction follows the rules of the protocol. For example, one local database system may commit its subtransaction and release locks, while the subtransaction at another local system is still executing. If the local systems permit control of locking behavior and all systems follow two-phase locking, then the federated database system can ensure that global transactions lock in a two-phase manner and the lock points of conflicting transactions would then define their global serialization order. If different local systems follow different concurrency-control mechanisms, however, this straightforward sort of global control does not work.

There are many protocols for ensuring consistency despite the concurrent execution of global and local transactions in federated database systems. Some are based on imposing sufficient conditions to ensure global serializability. Others ensure only a form of consistency weaker than serializability but achieve this consistency by less restrictive means.

There are several schemes to ensure global serializability in an environment where update transactions as well as read-only transactions can execute. Several of these schemes are based on the idea of a **ticket**. A special data item called a ticket is created in each local database system. Every global transaction that accesses data at a node must write the ticket at that node. This requirement ensures that global transactions conflict directly at every node they visit. Furthermore, the global transaction manager can control the order in which global transactions are serialized, by controlling the order in which the tickets are accessed. References to such schemes appear in the bibliographic notes for this chapter, available online.

23.6 Replication with Weak Degrees of Consistency

The replication protocols we have seen so far guarantee consistency, even if there are node and network failures. However, these protocols have a nontrivial cost, and further they may block if a significant number of nodes fail or get disconnected due to a network partition. Further, in the case of a network partition, a node that is not in the majority partition would not only be unable to perform writes, but it would also be unable to perform even reads.

Many applications wish to have higher availability, even at the cost of consistency. We study the trade-offs between consistency and availability in this section.

23.6.1 Trading Off Consistency for Availability

The protocols we have seen so far require a (weighted) majority of nodes be in a partition for updates to proceed. Nodes that are in a minority partition cannot process updates; if a network failure results in more than two partitions, no partition may have a majority of nodes. Under such a situation, the system would be completely unavailable for updates, and depending on the read-quorum, may even become unavailable for reads. The write-all-available protocol which we saw earlier provides availability but not consistency.

Ideally, we would like to have consistency and availability, even in the face of partitions. Unfortunately, this is not possible, a fact that is crystallized in the so-called **CAP theorem**, which states that any distributed database can have at most two of the following three properties:

- Consistency.
- Availability.
- Partition-tolerance.

The proof of the CAP theorem uses the following definition of consistency, with replicated data: an execution of a set of operations (reads and writes) on replicated data is said to be **consistent** if its result is the same as if the operations were executed on a single node, in a sequential order that is consistent with the ordering of operations issued by each process (transaction). The notion of consistency is similar to atomicity of transactions, but with each operation treated as a transaction, and is weaker than the atomicity property of transactions.

In any large-scale distributed system, partitions cannot be prevented, and as a result, either availability or consistency has to be sacrificed. The schemes we have seen earlier sacrifice availability for consistency in the face of partitions.

Consider a web-based social-networking system that replicates its data on three servers, and a network partition occurs that prevents the servers from communicating with each other. Since none of the partitions has a majority, it would not be possible to execute updates on any of the partitions. If one of these servers is in the same partition as a user, the user actually has access to data, but would be unable to update the data, since another user may be concurrently updating the same object in another partition, which could potentially lead to inconsistency. Inconsistency is not as great a risk in a social-networking system as in a banking database. A designer of such a system may decide that a user who can access the system should be allowed to perform updates on whatever replicas are accessible, even at the risk of inconsistency.

In contrast to systems such as banking databases that require the ACID properties, systems such as the social-networking system mentioned above are said to require the **BASE** properties:

- Basically available.
- Soft state.
- Eventually consistent.

The primary requirement is availability, even at the cost of consistency. Updates should be allowed, even in the event of partitioning, following, for example, the write-all-available protocol (which is similar to multimaster replication described in Section 23.6). Soft state refers to the property that the state of the database may not be precisely defined, with each replica possibly having a somewhat different state due to partitioning of the network. Eventually consistent is the requirement that once a partitioning is resolved, eventually all replicas will become consistent with each other.

This last step requires that inconsistent copies of data items be identified; if one is an earlier version of the other, the earlier version can be replaced by the later version. It is possible, however, that the two copies were the result of independent updates to a common base copy. A scheme for detecting such inconsistent updates, called the version-vector scheme, is described in Section 23.6.4.

Restoring consistency in the face of inconsistent updates requires that the updates be merged in some way that is meaningful to the application. We discuss possible solutions for resolution of conflicting updates, in Section 23.6.5.

In general, no system designer wants to deal with the possibility of inconsistent updates and the resultant problems of detection and resolution. Where possible, the system should be kept consistent. Inconsistent updates are allowed only when a node is disconnected from the network, in applications that can tolerate inconsistency.

Some key-value stores such as Apache Cassandra and MongoDB allow an application to specify how many replicas need to be accessible to carry out a write operation or a read operation. As long as a majority of replicas are accessible, there is no problem with consistency for writes. However, if the application sets the required number at less than a majority, and many replicas are inaccessible, updates are allowed to go ahead; there is, however, a risk of inconsistent updates, which must be resolved later.

For applications where inconsistency can cause significant problems, or is harder to resolve, system designers prefer to build fault-tolerant systems using replication and distributed consensus that avoid inconsistencies, even at the cost of potential non-availability.

23.6.2 Asynchronous Replication

Many relational database systems support replication with weak consistency, which can take one of several forms.

With **asynchronous replication** the database allows updates at a *primary* node (also referred to as the **master** node) and propagates updates to replicas at other nodes subsequently; the transaction that performs the update can commit once the update is performed at the primary, even before replicas are updated. Propagation of updates after commit is also referred to as **lazy propagation**. In contrast, the term **synchronous replication** refers to the case where updates are propagated to other replicas as part of a single transaction.

With asynchronous replication, the system must guarantee that once the transaction commits at the primary, the updates are eventually propagated to all replicas, even if there are system failures in between. Later in this section, we shall see how this property is guaranteed using persistent messaging.

Since propagation of updates is done asynchronously, a read at a replica may not get the latest version of a data item. Asynchronous propagation of updates is commonly used to allow update transactions to commit quickly, even at the cost of consistency. A system designer may choose to use replicas only for fault tolerance. However, if the replica is available on a local machine, or another machine that can be accessed with low latency, it may be much cheaper to read the data item at the replica instead of reading it from the primary, as long as the application is willing to accept potentially stale data values.

Data storage systems based on asynchronous replication may allow data items to have versions, with associated timestamps. A transaction may then request a version with required freshness properties, for example not more than 10 minutes old. If a local replica has a version of the data item satisfying the freshness criterion, it can be used; otherwise, the read may have to be sent to the primary node.

Consider, for example, an airline reservation site that shows the prices of multiple flight options. Prices may vary frequently, and the system does not guarantee that a user will actually be able to book a ticket at the price shown initially. Thus, it is quite acceptable to show a price that is a few minutes old. Asynchronous replication is a good solution for this application: price data can be replicated to a large number of servers, which share the load of user queries; and price data are updated at a primary node and replicated asynchronously to all other replicas.

Multiversion concurrency control schemes can be used to give a *transaction-consistent snapshot* of the database to read-only transactions that execute at a replica; that is, the transaction should see all updates of all transactions up to some transaction in the serialization order and should not see any updates of transactions later in the serialization order. The multiversion 2PL scheme, described in Section 23.5.1, can be extended to allow a read-only transaction to access a replica that may not have up-to-date versions of some data items, but still get a transaction-consistent snapshot view of the database. To do so, replicas must be aware of what is the latest timestamp t_{safe} such that they have received all updates with commit timestamp before t . Any read of a snapshot with timestamp $t < t_{safe}$ can be processed by that replica. Such a scheme is used in the Google Spanner database,

Asynchronous replication is used in traditional (centralized) databases to create one or more replicas of the database, on which large queries can be executed, without interfering with transactions running on a primary node. Such replication is referred to **master-slave replication**, since the replicas cannot perform any updates on their own but must only perform updates that the master node asks them to perform.

In such systems, asynchronous propagation of updates is typically done in a continuous fashion to minimize delays until an update is seen at a replica. However, in data warehouses, updates may be propagated periodically—every night, for example—so that update propagation does not interfere with query processing.

Some database systems support **multimaster replication** (also called **update-anywhere replication**); updates are permitted at any replica of a data item and are propagated to all replicas either synchronously, using two-phase commit, or asynchronously.

Asynchronous replication is also used in some distributed storage systems. Such systems partition data, as we have seen earlier, but replicate each partition. There is a primary node for each partition, and updates are typically sent to the primary node, which commits the updates locally, and propagates them asynchronously to the other replicas of the partition. Some systems such as PNUTS even allow each data item in a partition to specify which node should act as the primary node for that data item; that node is responsible for committing updates to the data item, and propagating the update to the other replicas. The motivation is to allow a node that is geographically close to a user to act as the primary node for data items corresponding to that user.

In any system supporting asynchronous propagation of updates, it is important that once an update is committed at the primary, it must definitely be delivered to the other replicas. If there are multiple updates at a primary node, they must be delivered in the same order to the replicas; out-of-order delivery can cause an earlier update to arrive late and overwrite a later update.

Persistent messaging, which we saw in Section 23.2.3, provides guaranteed delivery of messages and is widely used for asynchronous replication. The implementation techniques for persistent messages described in Section 23.2.3 can be easily modified to ensure that messages are delivered in the order in which they were sent. With persistent messaging, each primary node needs to be aware of the location of all the replicas.

Publish-subscribe systems, which we saw in Section 22.8.1, offer a more flexible way of ensuring reliable message delivery. Recall that publish-subscribe systems allow messages to be published with an associated topic, and subscribers can subscribe to any desired topic. To implement asynchronous replication, a topic is created corresponding to each partition. All replicas of a partition subscribe to the topic corresponding to the partition. Any update (including inserts, deletes, and data item updates) to a partition is published as a message with the topic corresponding to the partition. The publish-subscribe system ensures that once such a message is published, it will be delivered to all subscribers in the order in which it was published.

Publish-subscribe systems designed for parallel systems, such as the Apache Kafka system, or the Yahoo Message Bus service used for asynchronous replication in the PNUTS distributed data storage system, allow a large number of topics, and use mul-

multiple servers to handle messages to different topics in parallel. Thus, asynchronous replication can be made scalable.

Fault tolerance is an issue with asynchronous propagation of updates. If a primary node fails, a new node must take over as primary; this can be done either using an election algorithm, as we saw earlier or by having a master node (which is itself chosen by election) decide which node takes over the job of a failed primary node.

Consider what happens if a primary copy records an update but fails before the update is sent to the replicas. The new primary node has no way of finding out what was the last update committed at the primary copy. It can either wait for the primary to recover, which is unacceptable, or it can proceed without knowing what updates were committed just before failure. In the latter case, there is a risk that a transaction on the new primary may read an old value of a data item or perform an update that conflicts with an earlier update on the old primary.

To reduce the chance of such problems, some systems replicate the log records of the primary node to a backup node and allow the transaction to commit at the primary only after the log record has been successfully replicated at the backup node; if the primary node fails, the backup node takes over as the primary. Recall that this is the two-safe protocol from Section 19.7. This protocol is resilient to failure of one node, but not to the failure of two nodes.

If an application is built on top of a storage system using asynchronous replication, applications may potentially see some anomalous behaviors such as a read not seeing the effect of an earlier write done by the same application, or a later read seeing an earlier version of a data item than an earlier read, if different reads and writes are sent to different replicas. While such anomalies cannot be completely prevented in the event of failures, they can be avoided during normal operation by taking some precautions. For example, if read and write requests for a data item from a particular node are always sent to the same replica, the application will see any writes it has performed, and if two reads are performed on the same data item, the later read will see a version at least as new as the earlier read. This property is guaranteed if a primary replica is used to perform all actions on a data item.

23.6.3 Asynchronous View Maintenance

Indices and materialized views are forms of data derived from underlying data, and can thus be viewed as forms of replicated data. Just like replicas, indices and materialized views could be updated (maintained) as part of each transaction that updates the underlying data; doing so would ensure consistency of the derived data with the underlying data.

However, many systems prefer to perform index and view maintenance in an asynchronous manner, to reduce the overhead on transactions that update the underlying data. As a result, the indices and materialized views could be out of date. Any transac-

tion that uses such indices or materialized views must be aware that these structures may be out of date.

We now consider how to maintain indices and materialized views in the face of concurrent updates.

- The first requirement for view maintenance is for the subsystem that performs maintenance to receive information about updates to the underlying data in such a way that each update is delivered exactly once, despite failures.

Publish-subscribe systems are a good match for the first requirement above. All updates to any underlying relation are published to the pub-sub system with the relation name as the topic; the view maintenance subsystem subscribes to the topics corresponding to its underlying relations and received all relevant updates. As we saw in Section 22.8.1, we can have topics corresponding to each tablet of a stored relation. For a nonmaterialized intermediate relation that is partitioned, we can have a topic corresponding to each partition.

- The second requirement is for the subsystem to update the derived data in such a way that the derived data will be consistent with the underlying data, despite concurrent updates to the underlying data.

Since the underlying data may receive further updates as an earlier update is being processed, no asynchronous view maintenance technique can guarantee that the view state is consistent with the state of the underlying data at all times. However, the consistency requirement can be formalized as follows: if there are no updates to the underlying data for a sufficient amount of time, asynchronous maintenance must ensure that the derived data is consistent with the underlying data; such a requirement is known as an **eventual consistency** requirement.

The technique for parallel maintenance of materialized views which we saw in Section 22.7.5 uses the exchange operator model to send updates to nodes and allows view maintenance to be done locally. Techniques designed for view maintenance in a centralized setting can be used at each node, on locally materialized input data. Recall from Section 16.5.1 that view maintenance may be deferred, that is, it may be done after the transaction commits. Techniques for deferred view maintenance in a centralized setting already need to deal with concurrent updates; such techniques can be used locally at each node.

- A third requirement is for reads to get a consistent view of data. In general, a query that reads data from multiple nodes may not observe the updates of a transaction T on node N_1 , but may see the updates that T performed on node N_2 , thus seeing a transactionally inconsistent view of data. Systems that use asynchronous replication typically do not support transactionally consistent views of the database.

Further, scans of the database may not see an operation-consistent view of the database. (Recall the notion of operation consistency from Section 18.9, which requires that any operation should not see a database state that reflects only some of the updates of another operation. In Section 18.9 we saw an example of a scan

using an index that could see two versions, or neither version, of a record updated by a concurrent transaction, if the relation scan does not follow two-phase locking. A similar problem occurs with asynchronous propagation of updates, even if both the relation scan and the update transaction follow two-phase locking.

For example, consider a relation $r(A, B, C)$, with primary key A , which is partitioned on attribute B . Now consider a query that is scanning the relation r . Suppose there is a concurrent update to a tuple $t_1 \in r$, which updates attribute $t_1.B$ from v_1 to v_2 . Such an update requires deletion of the old tuple from the partition corresponding to value v_1 , and insertion of the new tuple in the partition corresponding to v_2 . These updates are propagated asynchronously.

Now, the scan of r could possibly scan the node corresponding to v_1 after the old tuple is deleted there but visit the node corresponding to v_2 before the asynchronous propagation inserts the updated tuple in that node. Then, the scan would completely miss the tuple, even though it should have seen either the old value or the new value of t_1 . Further, the scan could visit the node corresponding to v_1 before the delete is propagated to that node, and the node corresponding to v_2 after the insert is propagated to that node, and thereby see two versions of t_1 , one from before the update and one from after the update. Neither case would be possible with two-phase locking, if updates are propagated synchronously to all copies.

If a multiversion concurrency control technique is used, where data items have timestamps, snapshot reads are a good way to get a consistent scan of a relation; the snapshot timestamp should be set a sufficiently old value that all updates as of that timestamp have reached all replicas.

23.6.4 Detecting Inconsistent Updates

Many applications developed for such high availability are designed to continue functioning locally even when the node running the application is disconnected from the other nodes.

As an example, when data are replicated, and the network gets partitioned, if a system chooses to trade off consistency to get availability, updates may be done concurrently at multiple replicas. Such conflicting updates need to be detected and resolved. When a connection is re-established, the application needs to communicate with a storage system to send any updates done locally and fetch updates performed elsewhere. There is a potential for conflicting updates from different nodes. For example, node N_1 may update a locally cached copy of a data item while it is disconnected; concurrently another node may have updated the data item on the storage system, or may have updated its own local copy of the data item. Such conflicting updates must be detected, and resolved.

As another example, consider an application on a mobile device that supports offline updates (i.e., permits updates even if the mobile device is not connected to the

network). To give the user a seamless usage experience, such applications perform the updates on a locally cached copy, and then apply the update to the data store when the device goes back online. If the same data item may be updated from multiple devices, the problem of conflicting updates arises here, too. The schemes described below can be used in this context too, with nodes understood to also refer to mobile devices.

A mechanism for detecting conflicting updates is described in this section. How to resolve conflicting updates once they are detected is application dependent, and there is no general technique for doing so. However, some commonly used approaches are discussed in Section 23.6.5.

For data items updated by only one node, it is a simple matter to propagate the updates when the node gets reconnected to the storage system. If the node only caches read-only copies of data that may be updated by other nodes, the cached data may become inconsistent. When the node gets reconnected, it can be sent **invalidation reports** that inform it of out-of-date cache entries.

However, if updates can occur at more than one node, detecting conflicting updates is more difficult. Schemes based on **version numbering** allow updates of shared data from multiple nodes. These schemes do not guarantee that the updates will be consistent. Rather, they guarantee that, if two nodes independently update the same version of a data item, the clash will be detected eventually, when the nodes exchange information either directly or through a common node.

The **version-vector scheme** detects inconsistencies when replicas of a data item are independently updated. This scheme allows copies of a data item to be stored at multiple nodes.

The basic idea is for each node i to store, with its copy of each data item d , a **version vector**—that is, a set of version numbers $\{V[j]\}$, with one entry for each other node j on which the data item could potentially be updated. When a node i updates a data item d , it increments the version number $V[i]$ by one.

For example, suppose a data item is replicated at nodes N_1 , N_2 and N_3 . If the item is initially created at N_1 , the version vector could be $[1, 0, 0]$. If it is then replicated at N_2 , and then updated at node N_2 , the resultant version vector would be $[1, 1, 0]$. Suppose now that this version of the data item is replicated to N_3 , and then both N_2 and N_3 concurrently update the data item. Then, the version vector of the data item at N_2 would be $[1, 2, 0]$, while the version vector at N_3 would be $[1, 1, 1]$.

Whenever two nodes i and j connect with each other, they exchange updated data items, so that both obtain new versions of the data items. However, before exchanging data items, the nodes have to discover whether the copies are consistent:

1. If the version vectors V_i and V_j of the copy of the data item at nodes i and j are the same—that is, for each k , $V_i[k] = V_j[k]$ —then the copies of data item d are identical.
2. If, for each k , $V_i[k] \leq V_j[k]$ and the version vectors are not identical, then the copy of data item d at node i is older than the one at node j . That is, the copy of

data item d at node j was obtained by one or more modifications of the copy of the data item at node i . Node i replaces its copy of d , as well as its copy of the version vector for d , with the copies from node j .

In our example above, if N_1 had the vector $[1, 0, 0]$ for a data item, while N_2 had the vector $[1, 1, 0]$, then the version at N_2 is newer than the version at N_1 .

3. If there are a pair of values k and m such that $V_i[k] < V_j[k]$ and $V_i[m] > V_j[m]$, then the copies are *inconsistent*; that is, the copy of d at i contains updates performed by node k that have not been propagated to node j , and, similarly, the copy of d at j contains updates performed by node m that have not been propagated to node i . Then, the copies of d are inconsistent, since two or more updates have been performed on d independently.

In our example, after the concurrent updates at N_2 and N_3 , the two version vectors show the updates are inconsistent. Let V_2 and V_3 denote the version vectors at N_2 and N_3 . Then $V_2[2] = 2$ while $V_3[2] = 1$, whereas $V_2[3] = 0$, while $V_3[3] = 1$.

Manual intervention may be required to merge the updates. After merging the updates (perhaps manually), the version vectors are merged, by setting $V[k]$ to the maximum of $V_i[k]$ and $V_j[k]$ for each k . The node l that performs the write then increments $V[l]$ by 1 and then writes the data item and its version vector V .

The version-vector scheme was initially designed to deal with failures in distributed file systems. The scheme gained importance because mobile devices often store copies of data that are also present on server systems. The scheme is also widely used in distributed storage systems that allow updates to happen even if a node is not in a majority partition.

The version-vector scheme cannot solve the problem of how to reconcile inconsistent copies of data detected by the scheme. We discuss reconciliation in Section 23.6.5.

The version-vector scheme works well for detecting inconsistent updates to a single data item. However, if a storage system has a very large number of replicated items, finding which items have been inconsistently updated can be quite expensive if done naively. In Section 23.6.6 we study a data structure called a Merkle tree that can efficiently detect differences between sets of data items.

23.6.5 Resolving Conflicting Updates

Detection of conflicting updates may happen when a read operation fetches copies of a data item from multiple replicas or when the system executes a background process that compares data item versions.

At that point, conflicting updates on the same data item need to be **resolved**, to create a single common version. Resolution of conflicting updates is also referred to as **reconciliation**.

There is no technique for resolution so that can be used across all applications. We discuss some techniques that have been used in several commonly used applications.

Many applications can perform reconciliation automatically by executing on each node all update operations that had been performed on other nodes during a period of disconnection. This solution requires that the system keep track of operations, for example, adding an item to a shopping cart, or deleting an item from a shopping cart. This solution works if operations **commute**—that is, they generate the same result, regardless of the order in which they are executed. The addition of items to a shopping cart clearly commutes. Deletions do not commute with additions in general, which should be clear if you consider what happens if an addition of an item is exchanged with a delete of the same item. However, as long as deletion always operates only on items already present in the cart, this problem does not arise.

As another example, many banks allow customers to withdraw money from an ATM even if it is temporarily disconnected from the bank network. When the ATM gets reconnected, the withdrawal operation is applied to the account. Again, if there are multiple withdrawals, they may get merged in an order different from the order in which they happened in the real world, but the end result (balance) is the same. Note that since the operation already took place in the physical world, it cannot be rejected because of a negative balance; the fact that an account has a negative balance has to be dealt with separately.

There are other application-specific solutions for resolving conflicting updates. In the worst case, however, a system may need to alert humans to the conflicting updates, and let the humans decide how to resolve the conflict.

Dealing with such inconsistency automatically, and assisting users in resolving inconsistencies that cannot be handled automatically, remains an area of research.

23.6.6 Detecting Differences Between Collections Using Merkle Tree

The **Merkle tree** (also known as **hash tree**) is a data structure that allows efficient detection of differences between sets of data items that may be stored at different replicas. (To avoid confusion between tree nodes and system nodes, we shall refer to the latter as replicas in this section.)

Detecting items that have inconsistent values across replicas due to weak consistency is merely one of motivations for Merkle trees. Another motivation is performing sanity checks of replicas that are synchronously updated, and should be consistent, but may be inconsistent due to bugs or other failures. We consider below a binary version of the Merkle tree.

We assume that each data item has a key and a value; in case we are considering collections that do not have an explicit key, the data item value itself can be used as a key.

Each data item key k_i is hashed by a function $h_1()$ to get a hash value with n bits, where n is chosen such that 2^n is within a small factor of the number of data items. Each data item value v_i is hashed by another function $h_2()$ to get a hash value (which

is typically much longer than n bits). Finally, we assume a hash function $h_3()$ which takes as input a collection of hash values and returns a hash value computed from the collection (this hash function must be computed in a way that does not depend on the input order of the hash values, which can be done, for example, by sorting the collection before computing the hash function).

Each node of a Merkle tree has associated with it an identifier and stores a hash value. Each leaf of the tree can be identified by an n -bit binary number. For a given leaf identified by number k , consider the set of all data items i whose key k_i is such that $h_1(k_i) = k$. Then, the hash value v_k stored at leaf k is computed by applying $h_2()$ on each of the data item values v_i , and then applying $h_3()$ on the resultant collection of hash values. The system also maintains an index that can retrieve all the data items with a given hash value computed by function $h_2()$.

Figure 23.8 shows an example of a Merkle tree on 8 data items. The hash value of these data items on h_1 are shown on the left. Note that if for an item i_j , $h_1(i_j) = k$, then the data item i_j is associated with the leaf with identifier k .

Each internal node of the Merkle tree is identified by a hash value that is j bits long if the node is at depth j ; leaves are at depth n , and the root at depth 0. The internal node identified by a number k has as children nodes identified by $2k$ and $2k + 1$. The hash value stored v_k at node k is computed by applying $h_3()$ to the hash value stored at nodes $2k$ and $2k + 1$.

Now, suppose this Merkle tree is constructed on the data at two replicas (the replicas may be whole database replicas, or replicas of a partition of the database). If all items at the two replicas are identical, the stored hash values at the root nodes will also be identical.

As long as $h_2()$ computes a long enough hash value, and is suitably chosen, it is very unlikely that $h_2(v_1) = h_2(v_2)$ if $v_1 \neq v_2$, and similarly for $h_3()$. The SHA1 hash function with a 160-bit hash value is an example of a hash function that satisfies this

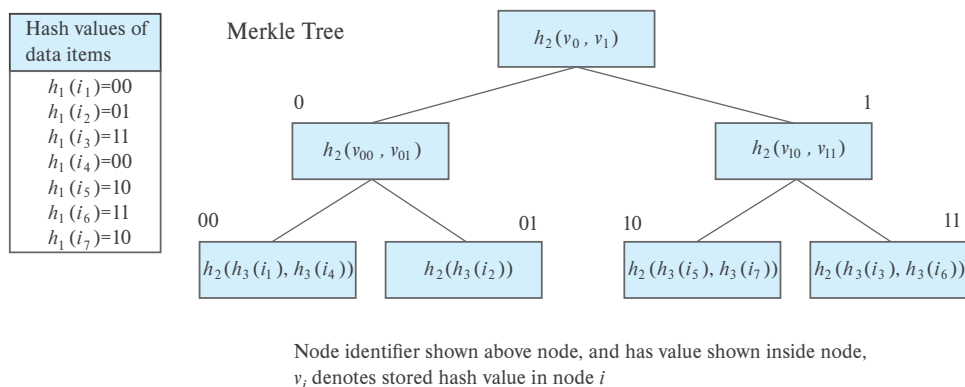


Figure 23.8 Example of Merkle tree.

requirement. Thus, we can assume that if two nodes have the same stored hash values, all the data items under the two nodes are identical.

If, in fact, there is a difference in the value of any items at the two replicas, or if an item is present at one replica but not at the other, the stored hash values at the root will be different, with high probability.

Then, the stored hash values at each of the children are compared with the hash values at the corresponding child in the other tree. Search traverses down each child whose hash value differs, until a leaf is reached. The traversal is done in parallel on both trees and requires communication to send tree node contents from one replica to the other.

At the leaf, if the hash values differ, the list of data item keys associated with the leaves, and the corresponding data item values are compared across the two trees, to find data items whose values differ as well as data items that are present in one of the trees but not in the other.

One such traversal takes time at most logarithmic in the number of leaf nodes of the tree; since the number of leaf nodes is chosen to be close to the number of data items, the traversal time is also logarithmic in the number of data items. This cost is paid at most once for each data item that differs between the two replicas. Furthermore, a path to a leaf is traversed only if there is, in fact, a difference at the leaf.

Thus, the overall cost for finding differences between two (potentially very large) sets is $O(m \log_2 N)$, where m is the number of data items that differ and N is the total number of data items. Wider trees can be used to reduce the number of nodes encountered in a traversal, which would be $\log_K N$ if each node has K children, at the cost of more data being transferred for each node. Wider trees are preferred if network latency is high compared to the network bandwidth.

Merkle trees have many applications; they can be used to find the difference in contents of two databases that are almost identical without transferring large amounts of data. Such inconsistencies can occur due to the use of protocols that only guarantee weak consistency. They could also occur because of message or network failures that result in differences in replicas, even if consensus or other protocols that guarantee consistent reads are used.

The original use of Merkle trees was for **verification of the contents** of a collection that may have potentially been corrupted by malicious users. Here, the Merkle tree leaf nodes must store the hash values of all data items that map to it, or a tree variant that only stores one data item at a leaf may be used. Further, the stored hash value at the root is digitally signed, meaning its contents cannot be modified by a malicious user who does not have the private key used for signing the value.

To check an entire relation, the hash values can be recomputed from the leaves upwards, and the recomputed hash value at the root can be compared with the digitally signed hash value stored at the root.

To check consistency of a single data item, its hash value is recomputed; and then so is the hash value for its leaf node n_i , using existing hash values for other data items

that hash to the same leaf. Next consider the parent node n_j of node n_i in the tree. The hash value of n_j is computed using the recomputed hash value of n_i and the already stored hash values of other children of n_j . This process is continued upward until the root of the tree. If the recomputed hash value at the root matches the signed hash value stored with the root, the contents of the data item can be determined to be uncorrupted.

The above technique works for detecting corruption since with suitably chosen hash functions, it is very hard for a malicious user to create replacement values for data items in a way that the recomputed hash value is identical to the signed hash value stored at the root.

23.7 Coordinator Selection

Several of the algorithms that we have presented require the use of a coordinator. If the coordinator fails because of a failure of the node at which it resides, the system can continue execution by restarting a new coordinator on another node. One way to continue execution is by maintaining a backup to the coordinator that is ready to assume responsibility if the coordinator fails. Another way is to “elect” a coordinator from among the nodes that are alive. We outline these options in this section. We then briefly describe fault-tolerant distributed services that have been developed to help developers of distributed applications perform these tasks.

23.7.1 Backup Coordinator

A **backup coordinator** is a node that, in addition to other tasks, maintains enough information locally to allow it to assume the role of coordinator with minimal disruption to the distributed system. All messages directed to the coordinator are received by both the coordinator and its backup. The backup coordinator executes the same algorithms and maintains the same internal state information (such as, for a concurrency coordinator, the lock table) as does the actual coordinator. The only difference in function between the coordinator and its backup is that the backup does not take any action that affects other nodes. Such actions are left to the actual coordinator.

In the event that the backup coordinator detects the failure of the actual coordinator, it assumes the role of coordinator. Since the backup has all the information available to it that the failed coordinator had, processing can continue without interruption.

The prime advantage of the backup approach is the ability to continue processing immediately. If a backup were not ready to assume the coordinator’s responsibility, a newly appointed coordinator would have to seek information from all nodes in the system so that it could execute the coordination tasks. Frequently, the only source of some of the requisite information is the failed coordinator. In this case, it may be necessary to abort several (or all) active transactions and to restart them under the control of the new coordinator.

Thus, the backup-coordinator approach avoids a substantial amount of delay while the distributed system recovers from a coordinator failure. The disadvantage is the overhead of duplicate execution of the coordinator's tasks. Furthermore, a coordinator and its backup need to communicate regularly to ensure that their activities are synchronized.

In short, the backup-coordinator approach incurs overhead during normal processing to allow fast recovery from a coordinator failure.

23.7.2 Election of Coordinator

In the absence of a designated backup coordinator, or in order to handle multiple failures, a new coordinator may be chosen dynamically by nodes that are live.

One possible approach is to have a designated node choose a new coordinator, when the current coordinator has failed. However, this raises the question of what to do if the node that chooses replacement coordinators itself fails.

If we have a fault-tolerant lock manager, a very effective way of choosing a new coordinator for a task is to use *lock leases*. The current coordinator has a lock lease on a data item associated with the task. If the coordinator fails, the lease will expire. If a participant determines that the coordinator may have failed, it attempts to get a lock lease for the task. Note that multiple participants may attempt to get a lease, but the lock manager ensures that only one of them can get the lease. The participant that gets the lease becomes the new coordinator. As discussed in Section 23.3.3, this ensures that only one node that can be the coordinator at a given time. Lock leases are widely used to ensure that a single node gets chosen as coordinator. However, observe that there is an underlying assumption of a fault-tolerant lock manager.

A participant determines that the coordinator may have failed if it is unable to communicate with the coordinator. Participants send periodic **heart-beat** messages to the coordinator and wait for an acknowledgment; if the acknowledgment is not received within a certain time, the coordinator is assumed to have failed.

Note that the participant cannot definitively distinguish a situation where the coordinator is dead from a situation where the network link between the node and the coordinator is cut. Thus, the system should be able to work correctly even if the current coordinator is alive, but another participant determines that the coordinator is dead. Lock leases ensure that at most one node can be the coordinator at any time; once a coordinator dies, another node can become the coordinator. However, lock leases work only if a fault-tolerant lock manager is available.

This raises the question of how to implement such a lock manager. We return later, in Section 23.8.4, to the question of how to implement a fault-tolerant lock manager. But it turns out that to do so efficiently, we need to have a coordinator. And, lock leases cannot be used to choose the coordinator for the lock manager! The problem of how to choose a coordinator without depending on a lock manager is solved by **election algorithms**, which enable the participating nodes to choose a new coordinator in a decentralized manner.

Suppose the goal is to elect a coordinator just once. Then, each node that wishes to become the coordinator proposes itself as a candidate to all the other nodes; such nodes are called *proposers*. The participating nodes then vote on which node among the candidates is to be chosen. If a majority of the participating nodes (called *acceptors*) vote for a particular candidate, it is chosen. A subset of nodes called *learners* ask the acceptor nodes for their vote and determine if a majority have voted for a particular candidate.

The problem with the above idea is that if there are multiple candidates, none of them may get a majority of votes. The question is what to do in such a situation. There are at least two approaches that have been proposed:

- Nodes are given unique numbers; if more than one candidate proposes itself, acceptors choose the highest-numbered candidate. Even then votes may be split with no majority decision, due to delayed or missing messages; in such a case, the election is run again. But if a node N_1 that was a candidate finds that a higher-numbered node N_2 has proposed itself as a candidate, then N_1 withdraws from the next round of the election. The highest-numbered candidate will win the election. The **bully algorithm** for election is based on this idea.

There are some subtle details due to the possibility that the highest-numbered candidate in one round may fail during a subsequent round, leading to there being no candidates at all! If a proposer observes that no coordinator was selected in a round where it withdrew itself as a candidate, it proposes itself as a candidate again in the next round.

Note also that the election has multiple rounds; each round has a number, and a candidate attaches a round number with the proposal. The round number is chosen to be the maximum round that it has seen, plus 1. A node can give a vote to only one candidate in a particular round, but it may change its vote in the next round.

- The second approach is based on *randomized retry*, which works as follows: If there is no majority decision in a particular round, all participants wait for a randomly chosen amount of time; if by that time a coordinator has been chosen by a majority of nodes, it is accepted as a coordinator. Otherwise, after the timeout, the node proposes itself as a candidate. As long as the timeouts are chosen properly (large enough compared to network latency) with high likelihood only one node proposes itself at a particular time and will get votes from a majority of nodes in a particular round.

If no candidate gets a majority vote in a round, the process is repeated. With very high probability, after a few rounds, one of the candidates gets a majority and is thus chosen as coordinator.

The randomized-retry approach was popularized by the Raft consensus algorithm, and it is easier to reason about it and show not just correctness, but also bounds on the expected time for an election round to succeed in choosing a coordinator, as compared to the node-numbering-based approach.

Note that the above description assumed that choosing a coordinator is a one-time activity. However, the chosen coordinator may fail, requiring a fresh election algorithm. The notion of a **term** is used to deal with this situation. As mentioned above, each time a node proposes itself as a coordinator, it associates the proposal with a round number, which is 1 more than the highest round number it has seen earlier, after ensuring that in the previous round no coordinator was chosen, or the chosen coordinator has subsequently failed. The round number is henceforth referred to as a **term**. When the election succeeds, the chosen coordinator is the coordinator for the corresponding term. If the election fails, the corresponding term does not have any coordinator chosen, but the election should succeed in a subsequent term.

Note also that there are subtle issues that arise since a node n may be disconnected from the network for a while, and it may get reconnected without ever realizing that it was disconnected. In the interim, the coordinator may have changed. In particular, if the node n was the coordinator, it may continue to think it is the coordinator, and some other node, say N_1 , which was also disconnected may think that n is still coordinator. However, if a coordinator was successfully elected, the majority of the nodes agree that some other node, say N_2 , is the coordinator.

In general, it is possible for more than one node to think that is the coordinator at the same time, although at most one of them can have the majority vote at that point in time.

To avoid this problem, each coordinator can be given a lease for a specified period. The coordinator can extend the lease by requesting an extension from other nodes and getting confirmation from a majority of the nodes. But if the coordinator is disconnected from a majority of the nodes, it cannot renew its lease, and the lease expires. A node can vote for a new coordinator only if the last lease time that it confirmed to the earlier coordinator has expired. Since a new coordinator needs a majority vote, it cannot get the vote until the lease time of the previous coordinator has expired.

However, even if leases are used to ensure that two nodes cannot be coordinators at the same time, delayed messages can result in a node getting a message from an old coordinator after a new one has been elected.

To deal with this problem, the current term of the sender is included with each message exchanged in the system. Note that when a node n is elected as coordinator, it has an associated term t ; participant nodes that learn that n is the coordinator are aware of the current term t . A node may receive a message with an old term either because an old coordinator did not realize it has been replaced or because of message delivery delay; the latter problem can occur even if leases or other mechanisms ensure that only one node can be the coordinator at a time. In either case, a node that receives a **stale message**, that is, one with a term older than the current term of the node, it can ignore the message. If a node receives a message with a higher number, it is behind the rest of the system, and it needs to find out the current term and coordinator by contacting other nodes.

Some protocols do not require the coordinator to store any state information; in such cases, the new coordinator can take over without any further actions. However,

other protocols require coordinators to retain state information. In such cases, the new coordinator has to reconstruct the state information from persistent data and recovery logs created by the previous coordinator. Such logs, in turn, need to be replicated to multiple nodes so that the loss of a node does not result in the loss of access to the recovery data. We shall see how to ensure availability by means of data replication in subsequent sections.

23.7.2.1 Distributed Coordination Services

There are a very large number of distributed applications that are in daily use today. Instead of each one having to implement its own mechanism for electing coordinators (among other tasks), it makes sense to develop a fault-tolerant coordination service that can be used by multiple distributed applications.

The **ZooKeeper** service is one such very widely used fault-tolerant distributed coordination service. The **Chubby** service developed earlier at Google is another such service, which is widely used for applications developed by Google. These services internally use consensus protocols to implement fault tolerance; we study consensus protocols in Section 23.8.

These services provide a file-system-like API, which supports the following features, among others:

- *Store (small amounts of) data* in files, with a hierarchical namespace. A typical use for such storage is to store configuration information that can be used to start up a distributed application, or for new nodes to join a distributed application by finding out which node is currently the coordinator.
- *Create and delete files*, which can be used to implement locking. For example, to get a lock, a process can attempt to create a file with a name corresponding to the lock. If another process has already created the file, the coordination service will return an error, so the process knows it could not get the lock.

For example, a node that acts as a master for a tablet in a key-value store would get a lock on a file whose name is the identifier of the tablet. This ensures that two nodes cannot be masters for the tablet at the same time.

If an overall application master detects that a tablet master has died, it could release the lock. If the service supports lock leases, this could happen automatically, if the tablet master does not renew its lease.

- *Watch for changes on a file*, which can be used by a process to check if a lock has been released, or to be informed about other changes in the system that require action by the process.

23.8 Consensus in Distributed Systems

In this section we first describe the consensus problem in a distributed system, that is, how a set of nodes agree on a decision in a fault-tolerant way. Distributed consensus is

a key building block for protocols that update replicated data in a fault-tolerant manner. We outline two consensus protocols, Paxos and Raft. We then describe replicated state machines, which can be used to make services, such as data storage systems and lock managers, fault tolerant. We end the section by describing how consensus can be used to make two-phase commit nonblocking.

23.8.1 Problem Overview

Software systems need to make decisions, such as the coordinator's decision on whether to commit or abort a transaction when using 2PC, or a decision on which node is to act as coordinator, in case a current coordinator fails.

If the decision is made by a single node, such as the commit/abort decision made by a coordinator node in 2PC, the system may block in case the node fails, since other nodes have no way of determining what decision was made. Thus, to ensure fault tolerance, multiple nodes must participate in the decision protocol; even if some of these nodes fail, the protocol must be able to reach a decision. A single node may make a proposal for a decision, but it must involve the other nodes to reach a decision in a fault-tolerant manner.

The most basic form of the **distributed consensus problem** is thus as follows: a set of n nodes (referred to as *participants*) need to agree on a decision by executing a protocol such that:

- All participants must “learn” the same value for the decision even if some nodes fail during the execution of the protocol, or messages are lost, or there are network partitions.
- The protocol should not block, and must terminate, as long as some majority of the nodes participating remain alive and can communicate with each other.

Any real system cannot just make a single decision once, but needs to make a series of decisions. A good abstract of the process of making multiple consensus decisions is to treat each decision as *adding a record to a log*. Each node has a copy of the log, and records are appended to the log at each node. There can potentially be conflicts on what record is added at what point in a log. The **multiple consensus protocol** viewed from this perspective needs to ensure that the log is uniquely defined.

Most consensus protocols allow temporary divergence of logs across nodes while the protocol is being executed; that is, the same log position at different nodes may have different records, and the end of the log may be different at different nodes. Shared-log consensus protocols keep track of an index into the log such that any entry before that index has definitely been agreed upon. Any entries after that index may be in the process of being agreed upon, or may be entries from failed attempts at consensus. However, the protocols subsequently bring the inconsistent parts of the log logs back in synchronization. To do this, log records at some nodes may be deleted after being inserted; such log records are viewed as not yet committed and cannot be used to make

decisions. Only log records in the prefix of the log that are in the committed prefix may be used to make decisions.

Several protocols have been proposed for distributed consensus. Of these, the Paxos family of protocols is one of the most popular, and it has been implemented in many systems. While the basic Paxos protocol is intuitively easy to understand at a high level, there are a number of details in its implementation that are rather complicated, and particularly so in the multiple consensus version. To address this issue, the Raft consensus protocol was developed, with ease of understanding and implementation being key goals, and it has been adopted by many systems. We outline the intuition behind these protocols in this section.

A key idea behind distributed consensus protocols is the idea of *voting* to make a decision; a particular decision succeeds only if a majority of the participating nodes vote for it. Note that if two or more different values are proposed for a particular decision, at most one of them can be voted for by a majority; thus, it is not possible for two different values to be chosen. Even if some nodes fail, if a majority of the participants vote for a value, it gets chosen, thus making the voting fault tolerant as long as a majority of the participants are up and connected to each other. There is, however, a risk that votes may get split between the proposed values, and some nodes may not vote if they fail; as a result, no value may be decided on. In such a case the voting procedure has to be executed again.

While the above intuition is easy enough to understand, there are many details that make the protocols nontrivial. We study some of these issues in the following sections.

We note that although we study some of the features of the Paxos and Raft consensus protocols, we omit a number of details that are needed for correct operation to keep our description concise.

We also note that a number of other consensus protocols have been proposed, and some of them are widely used, such as the Zab protocol which is part of the ZooKeeper distributed coordination service.

23.8.2 The Paxos Consensus Protocol

The basic Paxos protocol for making a single decision has the following participants.

1. One or more nodes that can propose a value for the decision; such nodes are called **proposers**.
2. One or more nodes that act as **acceptors**. An acceptor may get proposals with different values from different proposers and must choose (vote for) only one of the values.

Note that failure of an acceptor does not cause a problem, as long as a majority of the acceptors are live and reachable. Failure or disconnection of a majority would block the consensus protocol.

3. A set of nodes, called **learners**, query the acceptors to find what value each acceptor voted for in a particular round. (Acceptors could also send the value they accepted to the learners, without waiting for a query from the learner.)

Note that the same node can play the roles of proposer, acceptor, and learner.

If a majority of the acceptors voted for a particular value, that value is the chosen (consensus) value for that decision. But there are two problems:

1. It is possible for votes to be split among multiple proposals, and no proposal is accepted by a majority of the acceptors.

If any proposed value is to get a majority, at least some acceptors must change their decision. Thus, we must allow another round of decision making, where acceptors may choose a new value. This may need to be repeated as long as required until one value wins a majority vote.

2. Even if a majority of nodes do accept a value, it is possible that some of these nodes die or get disconnected after accepting a value, but before any learner finds out about their acceptance, and the remaining acceptors of that value do not constitute a majority.

If this is treated as the failure of a round, and a different value is chosen in a subsequent round, we have a problem. In particular, a learner that learned about the earlier majority would conclude that a particular value was chosen, while another learner could conclude that a different value was chosen, which is not acceptable.

Note also that acceptors must log their decision so when they recover they know what decision they made earlier.

The first problem above, namely, split votes, does not affect correctness, but it affects performance. To avoid this problem, Paxos makes use of a coordinator node. Proposers send a proposal to the coordinator, which picks one of the proposed values and follows the preceding steps to get a majority vote. If proposals come from only one coordinator, there is no conflict, and the lone proposed value gets a majority vote (modulo network and node failures).

Note that if the coordinator dies or is unreachable, a new coordinator can be elected, using techniques we saw earlier in Section 23.7, and the new coordinator can then do the same job as the earlier coordinator. Coordinators have no local state, so the new one can take over without any recovery steps.

The second problem, namely, different values getting majorities in different rounds, is a serious problem and must be avoided by the consensus protocol. To do so, Paxos uses the following steps:

1. Each proposal in Paxos has a number; different proposals must have different numbers.

2. In phase 1a of the protocol, a proposer sends a *prepare* message to acceptors, with its proposal number n .
3. In phase 1b of the protocol, an acceptor that receives a prepare message with number n checks if it has already responded to a message with a number higher than n . If so, it ignores the message. Otherwise, it remembers the number n and responds with the highest proposal number $m < n$ that it has already accepted, along with the corresponding value v ; if it has not accepted any value earlier, it indicates so in its response. (Note that responding is different from accepting.)
4. In phase 2a, the proposer checks if it got a response from a majority of the acceptors. If it does, it chooses a value v as follows: If none of the acceptors has already accepted any value, the proposer may use whatever value it intended to propose. If at least one of the acceptors responded that it accepted a value v with some number m , the proposer chooses the value v that has the highest associated number m (note that m must be $< n$).

The proposer now sends an *accept* request with the chosen value v and number n .

5. In phase 2b, when an acceptor gets an accept request with value v and number n , it checks if it has responded to a prepare message with number $n_1 > n$; if so it ignores the accept request. Otherwise, it accepts the proposed value v with number n .

The above protocol is quite clever, since it ensures the following: if a majority of acceptors accepted a value v (with any number n), then even if there are further proposals with number $n_1 > n$, the value proposed will be value v . Intuitively, the reason is that a value can be accepted with number n only if a majority of nodes respond to a prepare message with number n ; let us call this set of acceptors P . Suppose a value v had been accepted earlier by a majority of nodes with number m ; call this set of nodes A . Then A and P must have a node in common, and the common node will respond with value v and number m .

Note that some other proposal with a number $p > n$ may have been made earlier, but if it had been accepted by even one node, then a majority of nodes would have responded to the proposal with number p , and thus will not respond to the proposal with number n . Thus, if a proposal with value v is accepted by a majority of nodes, we can be sure that any further proposal will be for the already chosen value v .

Note that if a learner finds that no proposal was accepted by a majority of nodes, it can ask any proposer to issue a fresh proposal. If a value v had been accepted by a majority of nodes, it would be found and accepted again, and the learner would now learn about the value. If no value was accepted by a majority of nodes earlier, the new proposal could be accepted.

The above algorithm is for a single decision. Paxos has been extended to allow a series of decisions; the extended algorithm is called Multi-Paxos. Real implementations

also need to deal with other issues, such as how to add a node to the set of acceptors, or to remove a node from the set of acceptors if it is down for a long time, without affecting the correctness of the protocol. References with more details about Paxos and Multi-Paxos may be found in the bibliographic notes for this chapter, available online.

23.8.3 The Raft Consensus Protocol

There are several consensus protocols whose goal is to maintain a log, to which records can be appended in a fault-tolerant manner. Each node participating in such a protocol has a replica of the log. Log-based protocols simplify the handling of multiple decisions. The Raft consensus protocol is an example of such a protocol, and it was designed to be (relatively) easy to understand.

A key goal of log-based protocols is to keep the log replicas in sync by presenting a logical view of appending records atomically to all copies of the log. In fact, atomically appending the same entry to all replicas is not possible, due to failures. Recall that failure modes may include a node being temporarily disconnected and missing some updates, without ever realizing it was disconnected. Further, a log append may be done at just a few nodes, and the append process may fail subsequently, leaving other replicas without the record. Thus, ensuring all copies of the log are identical at all times is impossible. Such protocols must ensure the following:

- Even if a log replica is temporarily inconsistent with another, the protocol will bring it back in sync eventually by deleting and replacing log records on some copies.
- A log entry will not be treated as committed until the algorithm guarantees that it will never be deleted.

Protocols such as Raft that are based on log replication can allow each node to run a “state machine,” with log entries used as commands to the state machine; state machines are described in Section 23.8.4.

The Raft algorithm is based on having a coordinator, which is called a **leader** in Raft terminology. The other participating nodes are called **followers**. Since leaders may die and need to be replaced, time is divided into terms, which are identified by integers. Each term has a unique leader, although some terms may not have any associated leader. Later terms have higher identifiers than earlier terms.

Leaders are elected in Raft using the randomized-retry algorithm outlined in Section 23.7.2. Recall that the randomized-retry algorithm already incorporates the notion of a term. A node that votes for a leader does so for a specific term. Nodes keep track of the currentTerm based on messages from leaders or requests for votes.

Note that a leader N_1 may get temporarily disconnected, but get reconnected after other nodes find the leader cannot be reached, and elect a new leader N_2 . Node N_1

log index	1	2	3	4	5	6	7
leader	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	3 $z \leftarrow 4$
follower 1	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$		
follower 2	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	3 $z \leftarrow 4$
follower 3	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$				
follower 4	1 $x \leftarrow 2$	1 $z \leftarrow 2$	1 $x \leftarrow 3$	2 $x \leftarrow 4$	3 $x \leftarrow 1$	3 $y \leftarrow 6$	



committed entries

Figure 23.9 Example of Raft logs.

does not know that there is a new leader and may continue to execute the actions of a leader. The protocol should be robust to such situations.

Figure 23.9 shows an example of Raft logs at a leader and four followers. Note that the log index denotes the position of a particular record in a log. The number at the top of each log record is the term in which the log record was created, while the part below it shows the log entry, assumed here to record assignments to different variables.

Any node that wishes to append a record to the replicated log sends a log append request to the current leader. The leader adds its term as a field of the log records and appends the record to its log; it then sends an *AppendEntries* remote procedure call to the other nodes; the call contains several parameters, including these:

- *term*: the term of the current leader.
- *previousLogEntryPosition*: the position in the log of the preceding log entry.
- *previousLogEntryTerm*: the term associated with the preceding log entry.
- *logEntries*: an array of log records, allowing the call to append multiple log records at the same time.
- *leaderCommitIndex*: an index such that all log records at that index or earlier are committed. Recall that a log entry is not considered committed until a leader has confirmed that a majority of nodes have accepted that log entry. The leader keeps, in *leaderCommitIndex*, a position in the log such that all log records at that index and earlier are committed; this value is sent along with the *AppendEntries* call so that the nodes learn which log records have been committed.

If a majority of the nodes respond to the call with a return value true, the leader can report successful log append (along with the position in the log) to the node that initiated the log append. We will shortly see what happens if a majority do not respond with true.

Each follower that receives an AppendEntries message does the following:

1. If the term in the message is less than `currentTerm`, then Return false.
2. If the log does not contain an entry at a previous log entry position, whose term matches the term in the message, then Return false.
3. If there is an existing entry at the log position that is different from the first log record in the AppendEntries message, the existing entry and all subsequent log entries are deleted.
4. Any log records in the `logEntries` parameter that are not already in the log are appended to the log.
5. The follower also keeps track of a local `commitIndex` to track which records are committed. If the `leaderCommitIndex > commitIndex`, set `commitIndex = min(leaderCommitIndex, index of last entry in log)`.
6. Return true.

Note that the last step keeps track of the last committed log record. It is possible that the leader's log is ahead of the local log, so `commitIndex` cannot be blindly set to `leaderCommitIndex`, and it may need to be set to the local end of log if the `leaderCommitIndex` is ahead of the local end of log.

Figure 23.9 shows that different followers may have different log states, since some AppendEntries messages may not have reached those nodes. The part of the log up to entry 6 is present at a majority of nodes (namely, the leader, follower 2 and follower 4). On receipt of a true response to the AppendEntries call for the log record at position 6 from these followers, the leader can set `leaderCommitIndex` to 6.

It is possible for a node N_1 to be a leader in some term, and on temporary disconnection it may get replaced by a new leader N_2 in the next term. N_1 may not realize that there is a new leader for some time and may send appendEntry messages to other nodes. However, a majority of the other nodes will know about the new leader, and would have a term higher than that of N_1 . Thus, these nodes would return false, and include their current term in the response. Node N_1 would then realize that there is a leader with a new term; it then switches from the role of leader to that of follower.

The protocol must deal with the fact that some nodes may have outdated logs. Note that in step 2 of the follower protocol, the follower returns false if its log is outdated. In such a case, the leader will retry an AppendEntries, sending it all log records from an even earlier point in the log. This may happen several times, until the leader sends log

records from a point that is already in the follower log. At this point, the `AppendEntries` command would succeed.

A key remaining problem is to ensure that if a leader dies, and another one takes over, the log is brought to a consistent state. Note that the leader may have appended some log entries locally and replicated some of them to some other nodes, but the new leader may or may not have all these records. To deal with this situation, the Raft protocol includes steps to ensure the following:

1. The protocol ensures that any node elected as leader has all the committed log entries. To do so, any candidate must contact a majority of the nodes and send information about its log state when seeking a vote. A node will vote for a candidate in the election only if it finds that the candidate's log state is *at least as up-to-date* as its own; note that the definition of "at least as up-to-date" is a little complicated since it involves term identifiers in log records, and we omit details.

Since the above check is done by a majority of the nodes that voted for the new leader, any committed entry would certainly be present in the log of the newly elected leader.

2. The protocol then forces all other nodes to replicate the leader's log.

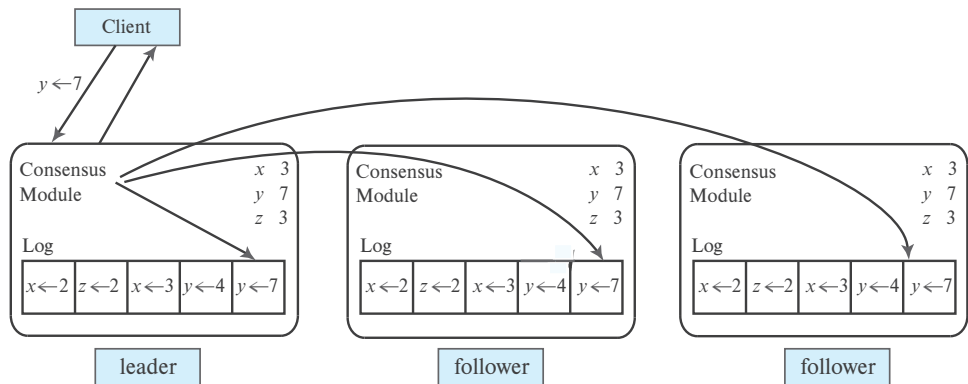
Note that the first step above does not actually find up to what log record is committed. Some of the log records at the new leader may not have been committed earlier, but may get committed when the new leader's log is replicated in this step.

There is also a subtle detail in that the new leader cannot count the number of replicas with a particular record from an earlier term, and declare it committed if it is at a majority of the nodes. Intuitively, the problem is because of the definition of "at least as up-to-date" and the possibility that a leader may fail, recover, and be elected as leader again. We omit details, but note that the way this problem is solved is for the new leader to replicate a new log record in its current term; when that log record is determined to be at a majority of the replicas, it and all earlier log records can be declared to be committed.

It should be clear that although the protocol, like Paxos, seems simple at a high level, there are many subtle details that need to be taken care of to ensure consistency even in the face of multiple failures and restarts. There are further details to be taken care of, including how to change the *cluster membership*, that is, the set of nodes that form the system, while the system is running (doing so carelessly can result in inconsistencies). Details of the above steps, including proofs of correctness, may be found in references in the bibliographic notes for this chapter, available online.

23.8.4 Fault-Tolerant Services Using Replicated State Machines

A key requirement in many systems is for a service to be made fault tolerant. A lock manager is an example of such a service, as is a key-value storage system.



Leader declares log record committed after it is replicated at a majority of nodes. Update of state machine at each replica happens only after log record has been committed.

Figure 23.10 Replicated state machine.

A very powerful approach to making services fault tolerant is to model them as “state machines” and then use the idea of replicated state machines that we describe next.

A **state machine** receives inputs and has a stored state; it makes state transitions on each input and may output some results along with the state transition. A **replicated state machine** is a state machine that is replicated on multiple nodes to make it fault tolerant. Intuitively, even if one of the nodes fails, the state and output can be obtained from any of the nodes that are alive, provided all the state machines are in a consistent state. The key to ensuring that the state machine replicas are consistent is to (a) require the state machines to be deterministic, and (b) ensure that all replicas get exactly the same input in the same order.

To ensure that all replicas get exactly the same input in the same order, we just append the inputs to a replicated log, using, for example, techniques we saw earlier in Section 23.8.3. As soon as a log entry is determined to be committed, it can be given as input to the state machine, which can then process it.

Figure 23.10 depicts a replicated state machine based on a replicated log. When a client issues a command, such as $y \leftarrow 7$ in the figure, the command is sent to the leader, where the command is appended to the log. The leader then replicates the command to the logs at the followers. Once a majority have confirmed that the command has been replicated in their logs, the leader declares the command committed and applies the command to its state machine. It also informs the followers of the commit, and the followers then apply the command to their state machine.

In the example in Figure 23.10, the state machine merely records the value of the updated variable; but in general, the state machine may execute any other actions. The actions are, however, required to be deterministic, so all state machines are in exactly

the same state when they have executed the same set of commands; the order of execution of commands will be the same since commands are executed in the log order.

Commands such as lock request must return a status to the caller. The status can be returned from any one of the replicas where the command is performed. Most implementations return the status from the leader node, since the request is sent to the leader, and the leader is also the first node to know when a log record has been committed (replicated to a majority of the nodes).

We now consider two applications that can be made fault-tolerant using the replicated state machine concept.

We first consider how to implement a **fault-tolerant lock manager**. A lock manager gets commands, namely, lock requests and releases, and maintains a state (lock table). It also gives output (lock grants or rollback requests on deadlock) on processing inputs (lock requests or releases). Lock managers can easily be coded to be deterministic, that is, given the same input, the state and output will be the same even if the code is executed again on a different node.

Thus, we can take a centralized implementation of a lock manager and run it on each node. Lock requests and releases are appended to a replicated log using, for example, the Raft protocol. Once a log entry is committed, the corresponding command (lock request or release) can be processed, in order, by the lock manager code at each replica. Even if some of the replicas fail, the other replicas can continue processing as long as a majority are up and connected.

Now consider the issue of implementing a **fault-tolerant key-value store**. A single-node storage system can be modeled as a state machine that supports `put()` and `get()` operations. The storage system is treated as a state machine, and the state machine is run on multiple nodes.

The `put()` operations are appended to the log using a consensus protocol and are processed when the consensus protocol declares the corresponding log records to be committed (i.e., replicated to a majority of the nodes).

If the consensus protocol uses leaders, `get()` operations need not be logged, and need to be executed only on the leader. To ensure that a `get()` operation sees the most recent `put()` on the same data item, all `put()` operations on the same data item that precede the `get()` operation in the log must be committed before the `get()` operation is processed. (If a consensus protocol does not use a leader, `get()` operations can also be logged and executed by at least one of the replicas which returns the value to the caller.)

Google's Spanner is an example of a system that uses the replicated state machine approach to creating a fault-tolerant implementation of a key-value storage system and a lock manager.

To ensure scalability, Spanner breaks up data into partitions, each of which has a subset of the data. Each partition has its data replicated across multiple nodes. Each node runs two state machines: one for the key-value storage system, and one for the lock manager. The set of replicas for a particular partition are called a *Paxos group*; one

of the nodes in a Paxos group acts as the Paxos group leader. Lock manager operations, as well as key-value store operations for a particular partition, are initiated at the Paxos group leader for that partition. The operations are appended to a log, which is replicated to the other nodes in the Paxos group using the Paxos consensus protocol.² Requests are applied in order at each member of the Paxos group, once they are committed.

As an optimization, `get()` operations are not logged, and executed only at the leader as described earlier. As a further optimization, Spanner allows reads to run as of a particular point in time, allowing reads to be executed at any replica of the partition (in other words, any other member of the Paxos group) that is sufficiently up to date, based on the multiversion two-phase locking protocol described earlier in Section 23.5.1.

23.8.5 Two-Phase Commit Using Consensus

Given a consensus protocol implementation, we can use it to create a **non-blocking two-phase commit** implementation. The idea is simple: instead of a coordinator recording its commit or abort decision locally, it uses a consensus protocol to record its decision in a replicated log. Even if the coordinator subsequently fails, other participants in the consensus protocol know about the decision, so the blocking problem is avoided.

In case the coordinator fails before making a decision for a transaction, a new coordinator can first check the log to see if a decision was made earlier, and if not it can make a commit/abort decision and use the consensus protocol to record the decision.

For example, in the Spanner system developed by Google, a transaction may span multiple partitions. Two-phase commit is initiated by a client and coordinated by the Paxos group leader at one of the partitions where the transaction executed. All other partitions where an update was performed acts as a participant in the two-phase commit protocol. Prepare and commit messages are sent to the Paxos group leader node of each of the partitions; recall that two-phase commit participants as well as coordinators record decisions in their local logs. These decisions are recorded by each leader, using consensus involving all the other nodes in its Paxos group.

If a Paxos group member other than the leader dies, the leader can continue processing the two-phase commit steps, as long as a majority of the group nodes are up and connected. If a Paxos group leader fails, one of the other group members takes over as the group leader. Note that all the state information required to continue commit processing is available to the new leader. Log records written during commit processing are available since the log is replicated. Also, recall from Section 23.8.4 that Spanner makes the lock manager fault tolerant by using the replicated state machine concept. Thus, a consistent replica of the lock table is also available with the new leader. Thus, the two-phase commit steps of both the coordinator and the participants can continue to be executed even if some nodes fail.

²The Multi-Paxos version of Paxos is used, but we shall just refer to it as Paxos for simplicity.